

EQUITY SELECTOR

Features & Targets Quant Reference

114 feature functions | 26 target functions

For each measure: brief purpose, physical interpretation, formula, pros, cons, required data, output columns and short Python.

Important: target columns contain future information and must never be included in the feature matrix used to predict them.

How to read the formulas

Notation	Meaning
P_t	closing price at time t
r_t	simple return $P_t/P_{(t-1)}-1$
w	lookback window
h	future target horizon
m / market	benchmark market return or price
σ	standard deviation / volatility
Q_q	empirical q quantile
Cross-sectional	computed across all stocks on the same date
Rolling	uses only current and past observations
Future	uses $t+1$ onward and therefore belongs only in targets

Validation note: forward horizons overlap, so walk-forward/purged validation is important. Cross-sectional and breadth measures also require a historically valid universe to avoid survivorship bias.

Package index

FEATURES

beta.py - *Market sensitivity and asymmetric beta features.* `rolling_beta()` | `downside_beta()` | `upside_beta()` | `beta_change()` | `beta_ratios()`

breadth.py - *Market participation across many stocks.* `breadth_ma()` | `advance_decline()` | `new_high_low()` | `positive_return_breadth()`

composite.py - *Combined multi-signal scores.* `composite_score()` | `momentum_score()` | `low_risk_score()` | `trend_score()`

correlation.py - *Benchmark co-movement and diversification state.* `rolling_correlation()` | `upside_downside_correlation()` | `correlation_change()`

cross_sectional.py - *Daily relative ranking and normalisation.* `cross_sectional_rank()` | `cross_sectional_z_score()` | `sector_neutral_rank()` | `winsorize_cross_section()`

dispersion.py - *Cross-sectional spread of outcomes and risk.* `return_dispersion()` | `momentum_dispersion()` | `volatility_dispersion()` | `drawdown_dispersion()`

distribution.py - *Rolling shape and dependence of return distributions.* `rolling_distribution()` | `rolling_quantiles()` | `rolling_iqr()` | `return_autocorrelation()`

drawdown.py - *Peak-relative loss depth, timing and duration.* `current_drawdown()` | `maximum_drawdown()` | `distance_from_high_low()` | `days_since_high_low()` | `drawdown_duration()` | `drawdown_change()`

experimental.py - *Persistence, entropy and random-walk diagnostics.* `hurst_exponent()` | `sign_entropy()` | `return_entropy()` | `variance_ratio()`

interactions.py - *Products that represent conditional signal relationships.* `feature_interactions()` | `momentum_volatility_interactions()` | `momentum_drawdown_interactions()` | `momentum_volume_interactions()`

liquidity.py - *Tradability, capacity and price-impact proxies.* `dollar_volume()` | `amihud_illiquidity()` | `turnover()` | `zero_return_frequency()`

market_relative.py - *Performance and trend relative to the broad market.* `market_relative_returns()` | `market_relative_strength()` | `market_relative_ma_distance()`

momentum.py - *Price continuation, acceleration and reversal measures.* `momentum()` | `skip_period_momentum()` | `momentum_change()` | `momentum_spread()` | `reversal()`

moving_averages.py - *Smoothed trend level, ratios and slopes.* `moving_average_distance()` | `exponential_moving_average_distance()` | `moving_average_ratios()` | `exponential_moving_average_ratios()` | `moving_average_slope()` | `exponential_moving_average_slope()`

ohlc.py - *Overnight and intraday path information from OHLC data.* `gap_features()` | `intraday_features()` | `candle_structure()` | `gap_volume_interaction()`

range_volatility.py - *Range-based realised volatility and movement size.* `high_low_range()` | `atr()` | `parkinson_volatility()` | `garman_klass_volatility()` | `rogers_satchell_volatility()` | `range_ratios()`

regimes.py - *Discrete/continuous market-state indicators.* `trend_regime()` | `volatility_regime()` | `drawdown_regime()` | `momentum_regime()`

residual.py - *Market-adjusted alpha, residual trend and idiosyncratic risk.* `rolling_alpha()` | `idiosyncratic_volatility()` | `residual_momentum()` | `market_r_squared()`

returns.py - *Lagged and rolling return primitives.* `return_lags()` | `log_return_lags()` | `absolute_return_lags()` | `squared_return_lags()` | `rolling_mean_return()` | `positive_negative_days()`

sector_relative.py - *Performance and trend relative to sector peers.* `sector_relative_returns()` | `sector_relative_strength()` | `sector_relative_ma_distance()`

tail_risk.py - *Historical downside-tail and extreme-return measures.* `historical_var()` | `expected_shortfall()` | `extreme_returns()` | `best_worst_return()` | `tail_ratio()`

technical.py - *Common statistical/technical transforms of price and range.* `rsi()` | `macd()` | `bollinger()` | `return_z_score()` | `stochastic_oscillator()`

trend.py - *Direction, smoothness and efficiency of price trends.* `trend_slope()` | `trend_r_squared()` | `trend_strength()` | `trend_efficiency()` | `trend_change()`

volatility.py - *Realised volatility level, asymmetry and persistence.* `rolling_volatility()` | `ewma_volatility()` | `volatility_ratios()` | `upside_downside_volatility()` | `semivariance()` | `volatility_change()` | `volatility_persistence()`

volume.py - *Trading-activity level, change and price-volume interactions.* `volume_changes()` | `average_volume()` | `relative_volume()` | `volume_momentum()` | `return_volume_interactions()`

TARGETS

barriers.py - *Path-dependent first-hit event labels.* `first_hit_barrier()` | `volatility_barrier()`

direction.py - *Directional and threshold classification labels.* `future_direction()` | `threshold_direction()` | `three_class_direction()`

drawdown.py - *Future downside-path targets.* `future_maximum_drawdown()` | `future_minimum_return()` | `future_drawdown_at_horizon()`

excursions.py - *Best/worst future path excursion and timing targets.* `maximum_favourable_excursion()` | `maximum_adverse_excursion()` | `time_to_favourable_excursion()` | `time_to_adverse_excursion()`

ranking.py - *Cross-sectional future-performance ranking labels.* `future_return_rank()` | `top_quantile_label()` | `bottom_quantile_label()`

returns.py - *Future return regression targets.* `forward_return()` | `forward_log_return()` | `forward_excess_return()`

risk_adjusted.py - *Future return adjusted for volatility or drawdown.* `future_return_volatility_ratio()` | `future_sortino_ratio()` | `future_return_minus_risk()` | `future_return_drawdown_ratio()`

volatility.py - *Future realised-risk targets.* `future_volatility()` | `future_variance()` | `future_upside_downside_volatility()` | `future_absolute_return()`

FEATURE REFERENCE

rolling_beta()

beta.py

Brief: Rolling sensitivity of the stock to benchmark returns.

Physical meaning: A beta above 1 means the stock has recently moved more than the market in the same direction; below 1 means less sensitivity.

Formula: $\beta_w = \text{Cov}_w(r_{\text{stock}}, r_{\text{market}}) / \text{Var}_w(r_{\text{market}})$

Pros: Widely used risk exposure measure; interpretable and comparable across stocks.

Cons: Benchmark- and window-dependent; beta can change quickly and misses nonlinear/tail behaviour.

Needs: Close + benchmark Close

Outputs: Rolling Beta {window}

Call: rolling_beta(df, windows=(20, 60, 120, 252), market_df=None, market_ticker='^GSPC')

Python: beta = r.rolling(w).cov(rm) / rm.rolling(w).var()

upside_beta()

beta.py

Brief: Beta estimated only on positive benchmark days.

Physical meaning: Shows how strongly the stock participates when the market rises.

Formula: $\beta_{up,w} = \text{Cov}(r_{\text{stock}}, r_m \mid r_m > 0) / \text{Var}(r_m \mid r_m > 0)$

Pros: Widely used risk exposure measure; interpretable and comparable across stocks.

Cons: Benchmark- and window-dependent; beta can change quickly and misses nonlinear/tail behaviour.

Needs: Close + benchmark Close

Outputs: Upside Beta {window}

Call: upside_beta(df, windows=(20, 60, 120, 252), market_df=None, market_ticker='^GSPC')

Python: rs = r.where(rm > 0); ms = rm.where(rm > 0)

beta = rs.rolling(w,min_periods=m).cov(ms) / ms.rolling(w,min_periods=m).var()

downside_beta()

beta.py

Brief: Beta estimated only on days when the benchmark return is negative.

Physical meaning: Shows how aggressively the stock tends to participate in market sell-offs.

Formula: $\beta_{down,w} = \text{Cov}(r_{\text{stock}}, r_m \mid r_m < 0) / \text{Var}(r_m \mid r_m < 0)$

Pros: Widely used risk exposure measure; interpretable and comparable across stocks.

Cons: Benchmark- and window-dependent; beta can change quickly and misses nonlinear/tail behaviour.

Needs: Close + benchmark Close

Outputs: Downside Beta {window}

Call: downside_beta(df, windows=(20, 60, 120, 252), market_df=None, market_ticker='^GSPC')

Python: rs = r.where(rm < 0); ms = rm.where(rm < 0)

beta = rs.rolling(w,min_periods=m).cov(ms) / ms.rolling(w,min_periods=m).var()

beta_change()

beta.py

Brief: Change in rolling beta over a chosen number of periods.

Physical meaning: Detects whether the stock is becoming more or less market-sensitive.

Formula: $\Delta \beta_{w,p,t} = \beta_{w,t} - \beta_{w,t-p}$

Pros: Widely used risk exposure measure; interpretable and comparable across stocks.

Cons: Benchmark- and window-dependent; beta can change quickly and misses nonlinear/tail behaviour.

Needs: Rolling beta or Close + benchmark

Outputs: Rolling Beta {window} Change {period}

Call: beta_change(df, windows=(20, 60, 120, 252), periods=(1, 5, 20), market_df=None, market_ticker='^GSPC')

Python: df[f"Rolling Beta {w} Change {p}"] = df[f"Rolling Beta {w}"].diff(p)

beta_ratios()

beta.py

Brief: Ratio of short-horizon beta to long-horizon beta.

Physical meaning: Values above 1 indicate recent market sensitivity is elevated relative to the longer-run baseline.

Formula: $BetaRatio(s,l) = \beta_s / \beta_l$

Pros: Widely used risk exposure measure; interpretable and comparable across stocks.

Cons: Benchmark- and window-dependent; beta can change quickly and misses nonlinear/tail behaviour.

Needs: Rolling beta or Close + benchmark

Outputs: Beta Ratio {short_window} {long_window}

Call: beta_ratios(df, short_windows=(20, 60), long_windows=(120, 252), market_df=None, market_ticker='^GSPC')

Python: df[f"Beta Ratio {s} {l}"] = df[f"Rolling Beta {s}"] / df[f"Rolling Beta {l}"]

advance_decline()

breadth.py

Brief: Cross-sectional balance between advancing and declining stocks.

Physical meaning: Shows whether daily/period market participation is tilted toward gains or losses.

Formula: $Diff_p = (Adv - Decl) / N$; $Ratio_p = Adv / Decl$

Pros: Captures whether a market move is broad or narrow rather than relying on an index alone.

Cons: Needs a stable multi-stock universe; survivorship and missing constituents can distort history.

Needs: Multi-stock Close matrix

Outputs: Advance Decline Difference {period}; Advance Decline Ratio {period}

Call: advance_decline(df, periods=(1,))

Python: ret = close.pct_change(p)

adv, dec = (ret>0).sum(1), (ret<0).sum(1)

breadth_ma()

breadth.py

Brief: Fraction of valid stocks trading above their own moving average.

Physical meaning: High breadth means a trend is supported by many stocks; low breadth means leadership is narrow.

Formula: $BreadthMA_w = \#(P_i > MA_{i,w}) / N_{valid}$

Pros: Captures whether a market move is broad or narrow rather than relying on an index alone.

Cons: Needs a stable multi-stock universe; survivorship and missing constituents can distort history.

Needs: Multi-stock Close matrix

Outputs: Breadth MA {window}

Call: breadth_ma(df, windows=(20, 50, 200))

Python: above = close > close.rolling(w).mean()

breadth = above.sum(axis=1) / valid.sum(axis=1)

new_high_low()

breadth.py

Brief: Share of stocks at rolling highs/lows and their net difference.

Physical meaning: Measures whether leadership is expanding into new highs or weakness is spreading into new lows.

Formula: $HighBreadth = \#new\ highs / N$; $LowBreadth = \#new\ lows / N$; $HL = (highs - lows) / N$

Pros: Captures whether a market move is broad or narrow rather than relying on an index alone.

Cons: Needs a stable multi-stock universe; survivorship and missing constituents can distort history.

Needs: Multi-stock Close matrix

Outputs: New High Breadth {window}; New Low Breadth {window}; High Low Breadth {window}

Call: new_high_low(df, windows=(20, 60, 252))

Python: hi = close.eq(close.rolling(w).max()).sum(1)

lo = close.eq(close.rolling(w).min()).sum(1)

positive_return_breadth()

breadth.py

Brief: Fraction of stocks with a positive return over a chosen horizon.

Physical meaning: Shows how widespread positive performance is across the universe.

Formula: $PosBreadth_w = \#(R_{i,w} > 0) / N_{valid}$

Pros: Captures whether a market move is broad or narrow rather than relying on an index alone.

Cons: Needs a stable multi-stock universe; survivorship and missing constituents can distort history.

Needs: Multi-stock Close matrix

Outputs: Positive Return Breadth {window}

Call: positive_return_breadth(df, windows=(5, 20, 60))

Python: ret = close.pct_change(w)
pos_breadth = (ret > 0).sum(1) / ret.notna().sum(1)

momentum_score()

composite.py

Brief: Composite of multiple momentum horizons.

Physical meaning: Summarises short-to-long trend strength in one number.

Formula: $MomentumScore = \text{weighted average of momentum features}$

Pros: Combines weak signals into a single score and can reduce reliance on one noisy input.

Cons: Weights and scales can be arbitrary; unstandardised inputs may let one feature dominate.

Needs: Momentum feature columns

Outputs: User-defined / wrapper output

Call: momentum_score(df, columns=('Momentum 20', 'Momentum 60', 'Momentum 120', 'Momentum 252'), weights=None)

Python: df = composite_score(df, columns, "Momentum Score", weights)

composite_score()

composite.py

Brief: Generic weighted average of selected feature columns, optionally sign-inverted.

Physical meaning: Builds one interpretable score from several signals pointing toward the same economic idea.

Formula: $Score = \sum_j (w_j * s_j * x_j) / \sum_j |w_j|$, where s_j is +/-1

Pros: Combines weak signals into a single score and can reduce reliance on one noisy input.

Cons: Weights and scales can be arbitrary; unstandardised inputs may let one feature dominate.

Needs: Selected feature columns

Outputs: name

Call: composite_score(df, columns, name, weights=None, invert=None)

Python: score = sum(x*w for x,w in zip(values,weights)) / sum(abs(w) for w in weights)

low_risk_score()

composite.py

Brief: Composite that inverts volatility, drawdown and beta-style inputs.

Physical meaning: Higher values are intended to represent lower recent risk exposure.

Formula: $LowRiskScore = \text{weighted average of -risk features}$

Pros: Combines weak signals into a single score and can reduce reliance on one noisy input.

Cons: Weights and scales can be arbitrary; unstandardised inputs may let one feature dominate.

Needs: Risk feature columns

Outputs: User-defined / wrapper output

Call: low_risk_score(df, columns=('Volatility 20', 'Drawdown 60', 'Rolling Beta 60'), weights=None)

Python: df = composite_score(df, columns, "Low Risk Score", weights, invert=list(columns))

trend_score()

composite.py

Brief: Composite of trend strength and efficiency measures.

Physical meaning: Higher values indicate a stronger and cleaner directional price path.

Formula: $TrendScore = weighted\ average\ of\ trend\ features$

Pros: Combines weak signals into a single score and can reduce reliance on one noisy input.

Cons: Weights and scales can be arbitrary; unstandardised inputs may let one feature dominate.

Needs: Trend feature columns

Outputs: User-defined / wrapper output

Call: trend_score(df, columns=('Trend Strength 20', 'Trend Strength 60', 'Trend Efficiency 20'), weights=None)

Python: df = composite_score(df, columns, "Trend Score", weights)

upside_downside_correlation()

correlation.py

Brief: Correlation calculated separately on positive and negative benchmark days.

Physical meaning: Reveals asymmetric diversification: a stock may decouple in rallies but become highly correlated in sell-offs.

Formula: $\rho_{up} = Corr(r_s, r_b \mid r_b > 0)$; $\rho_{down} = Corr(r_s, r_b \mid r_b < 0)$

Pros: Simple measure of co-movement useful for diversification and regime detection.

Cons: Correlation is unstable, nonlinear dependence is missed, and crisis correlation can differ sharply.

Needs: Close + benchmark Close

Outputs: Upside {label} Correlation {window}; Downside {label} Correlation {window}

Call: upside_downside_correlation(df, windows=(60, 252), benchmark_df=None, benchmark_ticker='^GSPC', label='Market')

Python: up = r.where(rb>0).rolling(w,min_periods=m).corr(rb.where(rb>0))
down = r.where(rb<0).rolling(w,min_periods=m).corr(rb.where(rb<0))

rolling_correlation()

correlation.py

Brief: Rolling Pearson correlation between stock and benchmark returns.

Physical meaning: Near +1 means they have recently moved together; near 0 means little linear co-movement.

Formula: $\rho_w = Corr_w(r_{stock}, r_{benchmark})$

Pros: Simple measure of co-movement useful for diversification and regime detection.

Cons: Correlation is unstable, nonlinear dependence is missed, and crisis correlation can differ sharply.

Needs: Close + benchmark Close

Outputs: {label} Correlation {window}

Call: rolling_correlation(df, windows=(20, 60, 120, 252), benchmark_df=None, benchmark_ticker='^GSPC', label='Market')

Python: corr = r.rolling(w).corr(rb)

correlation_change()

correlation.py

Brief: Difference in rolling correlation across time.

Physical meaning: Detects whether diversification against the benchmark is strengthening or disappearing.

Formula: $\Delta \rho_w(w, p, t) = \rho_w(w, t) - \rho_w(w, t - p)$

Pros: Simple measure of co-movement useful for diversification and regime detection.

Cons: Correlation is unstable, nonlinear dependence is missed, and crisis correlation can differ sharply.

Needs: Close + benchmark Close

Outputs: {label} Correlation {window} Change {period}

Call: correlation_change(df, windows=(20, 60, 252), periods=(5, 20), benchmark_df=None, benchmark_ticker='^GSPC', label='Market')

Python: corr = r.rolling(w).corr(rb)
df[name] = corr.diff(p)

cross_sectional_rank()

cross_sectional.py

Brief: Percentile rank of a feature across all stocks on the same date.

Physical meaning: Transforms an absolute signal into relative standing: e.g. 0.9 means top 10% of the universe.

Formula: $Rank_{i,t} = percentile_rank_i(x_{i,t})$

Pros: Directly supports stock ranking and makes features comparable across securities.

Cons: Results depend on the daily universe and outliers; ranks lose information about absolute magnitude.

Needs: Long-form multi-stock panel

Outputs: {column} {suffix}

Call: cross_sectional_rank(df, columns, date_col='Date', suffix='Rank')

Python: df[name] = df.groupby(date)[column].rank(pct=True)

sector_neutral_rank()

cross_sectional.py

Brief: Percentile rank within the stock's sector on each date.

Physical meaning: Identifies strength relative to direct sector peers instead of rewarding an entire hot sector.

Formula: $SectorRank_{i,t} = percentile_rank(x_{i,t} \mid sector_i)$

Pros: Directly supports stock ranking and makes features comparable across securities.

Cons: Results depend on the daily universe and outliers; ranks lose information about absolute magnitude.

Needs: Panel + sector labels

Outputs: {column} Sector Neutral Rank

Call: sector_neutral_rank(df, columns, sector_col='Sector', date_col='Date')

Python: df[name] = df.groupby([date, sector])[column].rank(pct=True)

cross_sectional_z_score()

cross_sectional.py

Brief: Daily cross-sectional standardisation of a feature.

Physical meaning: Shows how many cross-sectional standard deviations a stock sits above/below peers that day.

Formula: $z_{i,t} = (x_{i,t} - mean_t(x)) / std_t(x)$

Pros: Directly supports stock ranking and makes features comparable across securities.

Cons: Results depend on the daily universe and outliers; ranks lose information about absolute magnitude.

Needs: Long-form multi-stock panel

Outputs: {column} {suffix}

Call: cross_sectional_z_score(df, columns, date_col='Date', suffix='Cross Sectional Z Score')

Python: mean = g.transform("mean"); std = g.transform("std")

df[name] = (df[column]-mean)/std

winsorize_cross_section()

cross_sectional.py

Brief: Clips each day's feature values to chosen cross-sectional quantiles.

Physical meaning: Limits the influence of extreme observations without deleting the stock.

Formula: $x^*_{i,t} = clip(x_{i,t}, Q_{lower,t}, Q_{upper,t})$

Pros: Directly supports stock ranking and makes features comparable across securities.

Cons: Results depend on the daily universe and outliers; ranks lose information about absolute magnitude.

Needs: Long-form multi-stock panel

Outputs: {column} Winsorized

Call: winsorize_cross_section(df, columns, lower=0.01, upper=0.99, date_col='Date')

Python: lo=g.transform(lambda x:x.quantile(.01)); hi=g.transform(lambda x:x.quantile(.99))

df[name]=df[column].clip(lo,hi)

return_dispersio

dispersion.py

Brief: Cross-sectional standard deviation and IQR of stock returns.

Physical meaning: High dispersion means stock outcomes are spread widely, often creating more relative-value opportunity.

Formula: $Disp_p = Std_i(R_i, p); IQRDisp_p = Q75_i(R) - Q25_i(R)$

Pros: Measures cross-sectional opportunity and heterogeneity in the stock universe.

Cons: Can be driven by a few extreme stocks or sector composition; requires panel-wide data.

Needs: Multi-stock Close matrix

Outputs: Return Dispersion {period}; Return IQR Dispersion {period}

Call: return_dispersio(df, periods=(1, 5, 20))

Python: ret=close.pct_change(p)
std=ret.std(axis=1); iqr=ret.quantile(.75,axis=1)-ret.quantile(.25,axis=1)

volatility_dispersio

dispersion.py

Brief: Cross-sectional spread of rolling stock volatilities.

Physical meaning: High values mean risk is concentrated unevenly across stocks rather than uniformly high/low.

Formula: $VolDisp_w = Std_i(sigma_i, w)$

Pros: Measures cross-sectional opportunity and heterogeneity in the stock universe.

Cons: Can be driven by a few extreme stocks or sector composition; requires panel-wide data.

Needs: Multi-stock Close matrix

Outputs: Volatility Dispersion {window}

Call: volatility_dispersio(df, windows=(20, 60))

Python: vol=close.pct_change().rolling(w).std()
disp=vol.std(axis=1)

momentum_dispersio

dispersion.py

Brief: Cross-sectional spread of momentum across stocks.

Physical meaning: Shows whether recent winners and losers are far apart or moving similarly.

Formula: $MomDisp_w = Std_i(Momentum_i, w)$

Pros: Measures cross-sectional opportunity and heterogeneity in the stock universe.

Cons: Can be driven by a few extreme stocks or sector composition; requires panel-wide data.

Needs: Multi-stock Close matrix

Outputs: Momentum Dispersion {window}

Call: momentum_dispersio(df, windows=(20, 60, 120, 252))

Python: mom=close.pct_change(w)
disp=mom.std(axis=1)

drawdown_dispersio

dispersion.py

Brief: Cross-sectional spread of current drawdowns.

Physical meaning: Shows whether weakness is broad or concentrated in a subset of stocks.

Formula: $DDDisp_w = Std_i(P_i/High_i, w - 1)$

Pros: Measures cross-sectional opportunity and heterogeneity in the stock universe.

Cons: Can be driven by a few extreme stocks or sector composition; requires panel-wide data.

Needs: Multi-stock Close matrix

Outputs: Drawdown Dispersion {window}

Call: drawdown_dispersio(df, windows=(20, 60, 252))

Python: dd=close/close.rolling(w).max()-1
disp=dd.std(axis=1)

rolling_distribution()

distribution.py

Brief: Rolling skew, kurtosis, median, minimum and maximum of returns.

Physical meaning: Describes asymmetry, tail-heaviness, typical return and recent extremes.

Formula: $Skew_w$; $Kurt_w$; $Median_w$; Min_w ; Max_w over $r_{(t-w+1:t)}$

Pros: Describes return shape beyond mean and volatility, including asymmetry and tails.

Cons: Higher moments are noisy in short windows and can be dominated by a few observations.

Needs: Close

Outputs: Return Skew {window}; Return Kurtosis {window}; Return Median {window}; Return Minimum {window}; Return Maximum {window}

Call: rolling_distribution(df, windows=(20, 60, 252))

Python: roll=r.rolling(w)
skew=roll.skew(); kurt=roll.kurt(); med=roll.median(); lo=roll.min(); hi=roll.max()

rolling_iqr()

distribution.py

Brief: Interquartile range of recent returns.

Physical meaning: Measures typical dispersion while being less sensitive to extreme returns than standard deviation.

Formula: $IQR_w = Q_{0.75,w} - Q_{0.25,w}$

Pros: Describes return shape beyond mean and volatility, including asymmetry and tails.

Cons: Higher moments are noisy in short windows and can be dominated by a few observations.

Needs: Close

Outputs: Return IQR {window}

Call: rolling_iqr(df, windows=(20, 60, 252))

Python: iqr = r.rolling(w).quantile(.75) - r.rolling(w).quantile(.25)

rolling_quantiles()

distribution.py

Brief: Rolling empirical return percentiles.

Physical meaning: Shows the recent return level corresponding to chosen lower/upper parts of the distribution.

Formula: $Q_{q,w} = \text{empirical quantile}_q(r_{(t-w+1:t)})$

Pros: Describes return shape beyond mean and volatility, including asymmetry and tails.

Cons: Higher moments are noisy in short windows and can be dominated by a few observations.

Needs: Close

Outputs: Return Percentile {label} {window}

Call: rolling_quantiles(df, windows=(20, 60, 252), quantiles=(0.1, 0.25, 0.75, 0.9))

Python: q = r.rolling(w).quantile(level)

return_autocorrelation()

distribution.py

Brief: Rolling correlation between returns and lagged returns.

Physical meaning: Positive values suggest short-term persistence; negative values suggest reversal.

Formula: $ACF_w(w,k) = Corr_w(r_t, r_{(t-k)})$

Pros: Describes return shape beyond mean and volatility, including asymmetry and tails.

Cons: Higher moments are noisy in short windows and can be dominated by a few observations.

Needs: Close

Outputs: Return Autocorrelation {window} Lag {lag}

Call: return_autocorrelation(df, windows=(20, 60, 252), lags=(1, 2, 5))

Python: acf = r.rolling(w).corr(r.shift(k))

current_drawdown()

drawdown.py

Brief: Current percentage distance below the rolling peak.

Physical meaning: A value of -0.20 means price is 20% below the highest close in the lookback window.

Formula: $DD_w, t = P_t / \max(P_{(t-w+1:t)}) - 1$

Pros: Directly measures investor pain, path risk and position relative to prior peaks.

Cons: Backward-looking and path-dependent; a deep drawdown can signal either weakness or opportunity.

Needs: Close

Outputs: Drawdown {window}

Call: current_drawdown(df, windows=(20, 60, 120, 252))

Python: dd = df["Close"] / df["Close"].rolling(w).max() - 1

distance_from_high_low()

drawdown.py

Brief: Current distance from rolling high and rolling low.

Physical meaning: Shows where price sits inside its recent range: near highs, near lows, or between them.

Formula: $DistHigh = P / High_w - 1$; $DistLow = P / Low_w - 1$

Pros: Directly measures investor pain, path risk and position relative to prior peaks.

Cons: Backward-looking and path-dependent; a deep drawdown can signal either weakness or opportunity.

Needs: Close

Outputs: Distance From High {window}; Distance From Low {window}

Call: distance_from_high_low(df, windows=(5, 20, 60, 120, 252))

Python: hi=close.rolling(w).max(); lo=close.rolling(w).min()

dh=close/hi-1; dl=close/lo-1

maximum_drawdown()

drawdown.py

Brief: Worst peak-to-trough decline observed inside each rolling window.

Physical meaning: Measures the largest historical loss an investor could have experienced within that recent path.

Formula: $MDD_w = \min_{\tau}(P_{\tau} / \text{running_peak_}\tau - 1)$

Pros: Directly measures investor pain, path risk and position relative to prior peaks.

Cons: Backward-looking and path-dependent; a deep drawdown can signal either weakness or opportunity.

Needs: Close

Outputs: Maximum Drawdown {window}

Call: maximum_drawdown(df, windows=(20, 60, 120, 252))

Python: peak=np.maximum.accumulate(x)

mdd=np.min(x/peak-1)

days_since_high_low()

drawdown.py

Brief: Number of periods since the rolling-window high and low occurred.

Physical meaning: Distinguishes a fresh breakout/bottom from a peak/trough that occurred long ago.

Formula: $DaysSinceHigh = w - 1 - \text{argmax}(P_{\text{window}})$; *analogous for low*

Pros: Directly measures investor pain, path risk and position relative to prior peaks.

Cons: Backward-looking and path-dependent; a deep drawdown can signal either weakness or opportunity.

Needs: Close

Outputs: Days Since High {window}; Days Since Low {window}

Call: days_since_high_low(df, windows=(20, 60, 252))

Python: since_high = len(x)-1-np.argmax(x)

since_low = len(x)-1-np.argmin(x)

drawdown_duration()

drawdown.py

Brief: Number of consecutive periods the stock has remained below its running all-time high.

Physical meaning: Measures how long capital has been underwater rather than how deep the loss is.

Formula: $Duration_t = consecutive\ count\ of\ P_t < max_{(s \leq t)} P_s$

Pros: Directly measures investor pain, path risk and position relative to prior peaks.

Cons: Backward-looking and path-dependent; a deep drawdown can signal either weakness or opportunity.

Needs: Close

Outputs: Drawdown Duration

Call: drawdown_duration(df)

Python: running_high=close.cummax(); in_dd=close<running_high
duration=in_dd.groupby((~in_dd).cumsum()).cumsum()

hurst_exponent()

experimental.py

Brief: Rolling estimate of long-memory/persistence from scaling of price differences.

Physical meaning: Values above roughly 0.5 are often interpreted as persistence; below 0.5 as mean-reverting behaviour.

Formula: H is slope of $\log(std(P_{(t+k)}-P_t))$ versus $\log(k)$

Pros: Can capture persistence, randomness and complexity not represented by standard signals.

Cons: Estimates are noisy, parameter-sensitive and easier to overfit; treat as experimental evidence only.

Needs: Positive Close

Outputs: Hurst Exponent {window}

Call: hurst_exponent(df, windows=(60, 120, 252))

Python: tau=[np.std(x[k:]-x[:-k]) for k in lags]
H=np.polyfit(np.log(lags),np.log(tau),1)[0]

drawdown_change()

drawdown.py

Brief: Change in current drawdown over a chosen period.

Physical meaning: Positive change means recovery toward the peak; negative change means the drawdown is worsening.

Formula: $Delta\ DD_{(w,p,t)} = DD_{(w,t)} - DD_{(w,t-p)}$

Pros: Directly measures investor pain, path risk and position relative to prior peaks.

Cons: Backward-looking and path-dependent; a deep drawdown can signal either weakness or opportunity.

Needs: Close

Outputs: Drawdown {window} Change {period}

Call: drawdown_change(df, windows=(20, 60, 252), periods=(1, 5, 20))

Python: dd=close/close.rolling(w).max()-1
delta=dd.diff(p)

sign_entropy()

experimental.py

Brief: Entropy of positive/negative return signs.

Physical meaning: High entropy means up/down directions are balanced and unpredictable; low entropy means one sign dominates.

Formula: $H = -p \log p - (1-p) \log(1-p)$, $p = P(r > 0)$

Pros: Can capture persistence, randomness and complexity not represented by standard signals.

Cons: Estimates are noisy, parameter-sensitive and easier to overfit; treat as experimental evidence only.

Needs: Close

Outputs: Sign Entropy {window}

Call: sign_entropy(df, windows=(20, 60, 252))

Python: p=signs.mean()
entropy=- (p*np.log(p)+(1-p)*np.log(1-p))

return_entropy()

experimental.py

Brief: Histogram entropy of rolling returns.

Physical meaning: Higher values indicate returns are spread more evenly across states; lower values indicate concentration.

Formula: $H = -\sum_j p_j \log(p_j)$

Pros: Can capture persistence, randomness and complexity not represented by standard signals.

Cons: Estimates are noisy, parameter-sensitive and easier to overfit; treat as experimental evidence only.

Needs: Close

Outputs: Return Entropy {window}

Call: return_entropy(df, windows=(20, 60, 252), bins=10)

Python: counts, _ = np.histogram(x, bins=bins)
p = counts[counts > 0] / counts.sum(); H = -(p * np.log(p)).sum()

feature_interactions()

interactions.py

Brief: Pairwise product between user-selected feature columns.

Physical meaning: Represents conditional intensity: the effect is large only when both inputs are large in magnitude.

Formula: $Interaction = x_a * x_b$

Pros: Lets simple models represent conditional effects such as momentum behaving differently at high volatility.

Cons: Feature count grows quickly and products can amplify noise or scale differences.

Needs: Existing feature columns

Outputs: {left_feature} x {right_feature}

Call: feature_interactions(df, left_features, right_features)

Python: df[f"{a} x {b}"] = df[a] * df[b]

variance_ratio()

experimental.py

Brief: Compares multi-period return variance with scaled one-period variance.

Physical meaning: Values above 1 suggest persistence; below 1 can indicate mean reversion under a random-walk comparison.

Formula: $VR(k) = Var(\sum_{j=1}^k r_j) / (k Var(r))$

Pros: Can capture persistence, randomness and complexity not represented by standard signals.

Cons: Estimates are noisy, parameter-sensitive and easier to overfit; treat as experimental evidence only.

Needs: Close

Outputs: Variance Ratio {window} Lag {lag}

Call: variance_ratio(df, windows=(20, 60, 120), lags=(2, 5, 10))

Python: agg = r.rolling(k).sum()
vr = agg.rolling(w).var() / (k * r.rolling(w).var())

momentum_volatility_interactions()

interactions.py

Brief: Product of momentum and rolling volatility.

Physical meaning: Distinguishes the same trend occurring in calm conditions from one occurring in turbulent conditions.

Formula: $I(m, v) = Momentum_m * Volatility_v$

Pros: Lets simple models represent conditional effects such as momentum behaving differently at high volatility.

Cons: Feature count grows quickly and products can amplify noise or scale differences.

Needs: Close

Outputs: Momentum Volatility Interaction {momentum_window} {volatility_window}

Call: momentum_volatility_interactions(df, momentum_windows=(20, 60, 120), volatility_windows=(20, 60))

Python: interaction = close.pct_change(m) * r.rolling(v).std()

momentum_drawdown_interactions()

interactions.py

Brief: Product of momentum and current drawdown.

Physical meaning: Captures whether momentum is occurring near highs or while the stock remains deeply underwater.

Formula: $I(m, d) = \text{Momentum}_m * \text{Drawdown}_d$

Pros: Lets simple models represent conditional effects such as momentum behaving differently at high volatility.

Cons: Feature count grows quickly and products can amplify noise or scale differences.

Needs: Close

Outputs: Momentum Drawdown Interaction {momentum_window} {drawdown_window}

Call: momentum_drawdown_interactions(df, momentum_windows=(20, 60, 120), drawdown_windows=(20, 60, 252))

Python: interaction = close.pct_change(m) * (close/close.rolling(d).max()-1)

dollar_volume()

liquidity.py

Brief: Price multiplied by traded volume, plus rolling averages.

Physical meaning: Approximates the monetary amount traded and therefore the capacity available for entering/exiting positions.

Formula: $\text{DollarVolume}_t = P_t * \text{Volume}_t$

Pros: Relevant to tradability, market impact and whether apparent alpha can realistically be captured.

Cons: Daily volume is an imperfect proxy for execution cost; corporate actions and share-count data matter.

Needs: Close + Volume

Outputs: Dollar Volume; Average Dollar Volume 1; Average Dollar Volume {window}

Call: dollar_volume(df, windows=(1, 5, 20, 60))

Python: dv = df["Close"] * df["Volume"]
avg = dv.rolling(w).mean()

momentum_volume_interactions()

interactions.py

Brief: Product of momentum and relative volume.

Physical meaning: A trend confirmed by unusually heavy trading receives a different signal from the same move on quiet volume.

Formula: $I(m, v) = \text{Momentum}_m * \text{Volume} / \text{AvgVolume}_v$

Pros: Lets simple models represent conditional effects such as momentum behaving differently at high volatility.

Cons: Feature count grows quickly and products can amplify noise or scale differences.

Needs: Close + Volume

Outputs: Momentum Volume Interaction {momentum_window} {volume_window}

Call: momentum_volume_interactions(df, momentum_windows=(20, 60, 120), volume_windows=(20, 60))

Python: relvol=volume/volume.rolling(v).mean()
interaction=close.pct_change(m)*relvol

amihud_illiquidity()

liquidity.py

Brief: Average absolute return per unit of dollar volume.

Physical meaning: High values mean relatively large price moves occur for little traded value, suggesting lower liquidity.

Formula: $ILLIQ_w = \text{mean}_w(|r_t| / (P_t * V_t))$

Pros: Relevant to tradability, market impact and whether apparent alpha can realistically be captured.

Cons: Daily volume is an imperfect proxy for execution cost; corporate actions and share-count data matter.

Needs: Close + Volume

Outputs: Amihud Illiquidity {window}

Call: amihud_illiquidity(df, windows=(20, 60, 252))

Python: illiq=(r.abs()/((close*volume).replace(0,np.nan)).rolling(w).mean())

turnover()

liquidity.py

Brief: Trading volume relative to shares outstanding.

Physical meaning: Measures how quickly ownership is changing hands relative to company size.

Formula: $Turnover_t = Volume_t / SharesOutstanding$

Pros: Relevant to tradability, market impact and whether apparent alpha can realistically be captured.

Cons: Daily volume is an imperfect proxy for execution cost; corporate actions and share-count data matter.

Needs: Volume + shares outstanding

Outputs: Turnover 1; Turnover {window}

Call: turnover(df, shares_outstanding, windows=(1, 5, 20, 60))

Python: turnover = df["Volume"] / shares_outstanding
avg = turnover.rolling(w).mean()

market_relative_returns()

market_relative.py

Brief: Stock return minus benchmark return over the same horizon.

Physical meaning: Positive values mean the stock outperformed the market even if both rose or both fell.

Formula: $RelRet_p = R_{stock,p} - R_{market,p}$

Pros: Separates stock-specific strength from broad market direction.

Cons: Choice of market benchmark matters and relative signals can still contain sector effects.

Needs: Close + market Close

Outputs: Market Relative Return {period}

Call: market_relative_returns(df, market_df=None, market_ticker='^GSPC', periods=(1, 5, 20, 60, 120, 252))

Python: rel = close.pct_change(p) - market_close.pct_change(p)

zero_return_frequency()

liquidity.py

Brief: Fraction of days with exactly zero close-to-close return.

Physical meaning: Frequent zero-return days can indicate stale prices or very low trading activity.

Formula: $ZeroFreq_w = mean_w(1[r_t=0])$

Pros: Relevant to tradability, market impact and whether apparent alpha can realistically be captured.

Cons: Daily volume is an imperfect proxy for execution cost; corporate actions and share-count data matter.

Needs: Close

Outputs: Zero Return Frequency {window}

Call: zero_return_frequency(df, windows=(20, 60, 252))

Python: zero=(close.pct_change()==0).astype(float)
freq=zero.rolling(w).mean()

market_relative_strength()

market_relative.py

Brief: Momentum of the stock-to-market price ratio.

Physical meaning: Measures whether the stock has been gaining or losing ground against the market.

Formula: $RS_w = (P_{stock}/P_{market})_t / (P_{stock}/P_{market})_{(t-w)} - 1$

Pros: Separates stock-specific strength from broad market direction.

Cons: Choice of market benchmark matters and relative signals can still contain sector effects.

Needs: Close + market Close

Outputs: Market Relative Strength {window}

Call: market_relative_strength(df, market_df=None, market_ticker='^GSPC', windows=(20, 60, 120, 252))

Python: relative_price=close/market_close
rs=relative_price.pct_change(w)

market_relative_ma_distance()

market_relative.py

Brief: Difference between stock and market distances from their moving averages.

Physical meaning: Shows whether the stock is more extended/stronger relative to its own trend than the market is.

Formula: $RelMA_{Dist_w} = (P_s/MA_s, w-1) - (P_m/MA_m, w-1)$

Pros: Separates stock-specific strength from broad market direction.

Cons: Choice of market benchmark matters and relative signals can still contain sector effects.

Needs: Close + market Close

Outputs: Market Relative MA Distance {window}

Call: market_relative_ma_distance(df, market_df=None, market_ticker='^GSPC', windows=(20, 50, 200))

Python: rel=(close/close.rolling(w).mean()-1) - (market/market.rolling(w).mean()-1)

skip_period_momentum()

momentum.py

Brief: Longer-horizon momentum that excludes the most recent part of the lookback.

Physical meaning: Separates medium-term trend from very recent reversal/noise.

Formula: $SkipMom_w(s) = P_t(s) / P_t(w) - 1$

Pros: One of the most established systematic equity signal families; easy to interpret and rank.

Cons: Can reverse sharply, crowding can hurt, and short horizons may be dominated by noise.

Needs: Close

Outputs: Momentum {window} Skip {skip_period}

Call: skip_period_momentum(df, windows=(60, 120, 252), skip_periods=(5, 20))

Python: skip_mom = close.shift(s) / close.shift(w) - 1

momentum()

momentum.py

Brief: Cumulative percentage price change over each lookback window.

Physical meaning: Positive momentum means the stock has appreciated over the window; magnitude measures trend strength.

Formula: $Mom_w, t = P_t / P_t(w) - 1$

Pros: One of the most established systematic equity signal families; easy to interpret and rank.

Cons: Can reverse sharply, crowding can hurt, and short horizons may be dominated by noise.

Needs: Close

Outputs: Momentum {window}

Call: momentum(df, windows=(2, 3, 5, 10, 20, 40, 60, 90, 120, 180, 252))

Python: df[f"Momentum {w}"] = df["Close"].pct_change(w)

momentum_change()

momentum.py

Brief: Change in a momentum measure over time.

Physical meaning: Shows whether an existing trend is strengthening or weakening.

Formula: $Delta_Mom_w(p, t) = Mom_w(t) - Mom_w(t-p)$

Pros: One of the most established systematic equity signal families; easy to interpret and rank.

Cons: Can reverse sharply, crowding can hurt, and short horizons may be dominated by noise.

Needs: Close

Outputs: Momentum {window} Change {period}

Call: momentum_change(df, windows=(5, 20, 60, 120), periods=(1, 5, 20))

Python: mom=close.pct_change(w)

change=mom.diff(p)

momentum_spread()

momentum.py

Brief: Difference between short- and long-horizon momentum.

Physical meaning: Positive spread indicates recent performance is stronger than the longer-term trend.

Formula: $Spread(s, l) = Mom_s - Mom_l$

Pros: One of the most established systematic equity signal families; easy to interpret and rank.

Cons: Can reverse sharply, crowding can hurt, and short horizons may be dominated by noise.

Needs: Close

Outputs: Momentum Spread {short_window} {long_window}

Call: momentum_spread(df, short_windows=(5, 20, 60), long_windows=(20, 60, 120, 252))

Python: spread=close.pct_change(s)-close.pct_change(l)

moving_average_distance()

moving_averages.py

Brief: Percentage distance of price from a simple moving average.

Physical meaning: Positive values mean price is above its recent average; magnitude measures trend extension.

Formula: $MADist_w = P_t / SMA_w - 1$

Pros: Provides smooth trend/state measures and reduces day-to-day noise.

Cons: Lagging by construction; many variants are highly correlated and can add redundant features.

Needs: Close

Outputs: Distance From MA {window}

Call: moving_average_distance(df, windows=(5, 10, 20, 50, 100, 200))

Python: ma=close.rolling(w).mean()

dist=close/ma-1

reversal()

momentum.py

Brief: Negative of recent cumulative return.

Physical meaning: Turns recent losers into positive signals and recent winners into negative signals for mean-reversion research.

Formula: $Reversal_w = -R_w$

Pros: One of the most established systematic equity signal families; easy to interpret and rank.

Cons: Can reverse sharply, crowding can hurt, and short horizons may be dominated by noise.

Needs: Close

Outputs: Reversal {window}

Call: reversal(df, windows=(1, 2, 5, 10))

Python: reversal = -df["Close"].pct_change(w)

exponential_moving_average_distance()

moving_averages.py

Brief: Percentage distance from an exponentially weighted moving average.

Physical meaning: Like MA distance but reacts faster because recent prices carry more weight.

Formula: $EMADist_s = P_t / EMA_s - 1$

Pros: Provides smooth trend/state measures and reduces day-to-day noise.

Cons: Lagging by construction; many variants are highly correlated and can add redundant features.

Needs: Close

Outputs: Distance From EMA {span}

Call: exponential_moving_average_distance(df, spans=(5, 10, 20, 50, 100, 200))

Python: ema=close.ewm(span=s,adjust=False).mean()

dist=close/ema-1

moving_average_ratios()

moving_averages.py

Brief: Relative level of a short SMA to a longer SMA.

Physical meaning: Positive values indicate the shorter trend is above the longer trend, a smooth trend-strength signal.

Formula: $MARatio(s, l) = SMA_s / SMA_l - 1$

Pros: Provides smooth trend/state measures and reduces day-to-day noise.

Cons: Lagging by construction; many variants are highly correlated and can add redundant features.

Needs: Close

Outputs: MA Ratio {short_window} {long_window}

Call: moving_average_ratios(df, short_windows=(5, 10, 20, 50, 100), long_windows=(20, 50, 100, 200))

Python: ratio=close.rolling(s).mean()/close.rolling(l).mean()-1

moving_average_slope()

moving_averages.py

Brief: Percentage change in an SMA over a chosen period.

Physical meaning: Measures whether the smoothed trend itself is rising or falling and how quickly.

Formula: $MASlope(w, p) = SMA_w, t / SMA_w, t - p - 1$

Pros: Provides smooth trend/state measures and reduces day-to-day noise.

Cons: Lagging by construction; many variants are highly correlated and can add redundant features.

Needs: Close

Outputs: MA {window} Slope {period}

Call: moving_average_slope(df, windows=(20, 50, 100, 200), periods=(1, 5, 20))

Python: ma=close.rolling(w).mean()

slope=ma.pct_change(p)

exponential_moving_average_ratios()

moving_averages.py

Brief: Relative level of short and long EMAs.

Physical meaning: Captures fast-versus-slow trend state with more emphasis on recent prices.

Formula: $EMARatio(s, l) = EMA_s / EMA_l - 1$

Pros: Provides smooth trend/state measures and reduces day-to-day noise.

Cons: Lagging by construction; many variants are highly correlated and can add redundant features.

Needs: Close

Outputs: EMA Ratio {short_span} {long_span}

Call: exponential_moving_average_ratios(df, short_spans=(5, 10, 20, 50), long_spans=(20, 50, 100, 200))

Python: ratio=close.ewm(span=s, adjust=False).mean()/close.ewm(span=l, adjust=False).mean()-1

exponential_moving_average_slope()

moving_averages.py

Brief: Percentage change in an EMA over a chosen period.

Physical meaning: A faster-reacting estimate of change in the smoothed trend.

Formula: $EMASlope(s, p) = EMA_s, t / EMA_s, t - p - 1$

Pros: Provides smooth trend/state measures and reduces day-to-day noise.

Cons: Lagging by construction; many variants are highly correlated and can add redundant features.

Needs: Close

Outputs: EMA {span} Slope {period}

Call: exponential_moving_average_slope(df, spans=(20, 50, 100, 200), periods=(1, 5, 20))

Python: ema=close.ewm(span=s, adjust=False).mean()

slope=ema.pct_change(p)

gap_features()

ohlc.py

Brief: Overnight open-versus-prior-close gap plus rolling gap statistics.

Physical meaning: Separates information arriving outside regular trading hours and tells whether today's gap is unusual.

Formula: $Gap_t = Open_t / Close_t - 1$; $z = (Gap - mean_w) / std_w$

Pros: Uses information inside the trading day and overnight session that close-only features miss.

Cons: Daily OHLC compresses intraday path information and can be noisy around corporate/news events.

Needs: Open + Close

Outputs: Overnight Gap; Absolute Overnight Gap; Mean Gap {window}; Gap Volatility {window}; Gap Z Score {window}

Call: gap_features(df, windows=(5, 20, 60))

Python: gap=open_/close.shift(1)-1
z=(gap-gap.rolling(w).mean())/gap.rolling(w).std()

candle_structure()

ohlc.py

Brief: Normalised candle body, direction and wick sizes.

Physical meaning: Quantifies whether the day was directional or indecisive and whether price rejected highs/lows.

Formula: $BodyRatio = |C - O| / (H - L)$; $UpperWick = (H - \max(O, C)) / (H - L)$; $LowerWick = (\min(O, C) - L) / (H - L)$

Pros: Uses information inside the trading day and overnight session that close-only features miss.

Cons: Daily OHLC compresses intraday path information and can be noisy around corporate/news events.

Needs: OHLC

Outputs: Candle Body Range Ratio; Candle Direction; Upper Wick Range Ratio; Lower Wick Range Ratio

Call: candle_structure(df)

Python: rng=high-low
body=abs(close-open)/rng; upper=(high-np.maximum(open_,close))/rng

intraday_features()

ohlc.py

Brief: Open-to-close, open-to-extreme, extreme-to-close and close-location measures.

Physical meaning: Describes where the day travelled and where it finished inside its intraday range.

Formula: $Intraday = Close / Open - 1$; $CLV = (Close - Low) / (High - Low)$

Pros: Uses information inside the trading day and overnight session that close-only features miss.

Cons: Daily OHLC compresses intraday path information and can be noisy around corporate/news events.

Needs: OHLC

Outputs: Intraday Return; Open To High; Open To Low; High To Close; Low To Close; Close Location Value

Call: intraday_features(df)

Python: intraday=close/open_-1
clv=(close-low)/(high-low)

gap_volume_interaction()

ohlc.py

Brief: Overnight gap multiplied by relative volume.

Physical meaning: A large gap accompanied by unusually strong participation is separated from an unconfirmed gap.

Formula: $GapVol_w = Gap_t * Volume_t / AvgVolume_w$

Pros: Uses information inside the trading day and overnight session that close-only features miss.

Cons: Daily OHLC compresses intraday path information and can be noisy around corporate/news events.

Needs: OHLCV

Outputs: Gap Volume Interaction {window}

Call: gap_volume_interaction(df, volume_windows=(20, 60))

Python: gap=open_/close.shift(1)-1
interaction=gap*(volume/volume.rolling(w).mean())

high_low_range()

range_volatility.py

Brief: Normalised daily high-low range and rolling range statistics.

Physical meaning: Measures intraday movement size independently of direction.

Formula: $Range_t = (High_t - Low_t) / Close_t$

Pros: Uses high/low/open information to estimate intraday variability more efficiently than close-only measures.

Cons: Sensitive to bad OHLC prints, overnight jumps and model assumptions; estimators are not interchangeable.

Needs: High + Low + Close

Outputs: High Low Range; Mean High Low Range {window}; High Low Range Volatility {window}; High Low Range Z Score {window}

Call: high_low_range(df, windows=(5, 20, 60))

Python: rng=(df["High"]-df["Low"])/df["Close"]

parkinson_volatility()

range_volatility.py

Brief: Range-based volatility estimator using high and low prices.

Physical meaning: Uses intraday extremes to estimate variability more efficiently when prices are continuously observed.

Formula: $\sigma_P = \sqrt{\text{mean}(\log(H/L)^2) / (4 \log 2)}$

Pros: Uses high/low/open information to estimate intraday variability more efficiently than close-only measures.

Cons: Sensitive to bad OHLC prints, overnight jumps and model assumptions; estimators are not interchangeable.

Needs: High + Low

Outputs: Parkinson Volatility {window}

Call: parkinson_volatility(df, windows=(20, 60, 120))

Python: sigma=np.sqrt((np.log(high/low)**2).rolling(w).mean()/(4*np.log(2)))

atr()

range_volatility.py

Brief: Average True Range and ATR as a percentage of price.

Physical meaning: Captures typical trading range including overnight gaps; ATR% makes stocks comparable across price levels.

Formula: $TR = \max(H-L, |H-C_{prev}|, |L-C_{prev}|)$; $ATR_w = \text{mean}_w(TR)$

Pros: Uses high/low/open information to estimate intraday variability more efficiently than close-only measures.

Cons: Sensitive to bad OHLC prints, overnight jumps and model assumptions; estimators are not interchangeable.

Needs: High + Low + Close

Outputs: ATR {window}; ATR Percent {window}

Call: atr(df, windows=(5, 10, 14, 20, 60))

Python: tr=np.maximum(high-low,np.maximum(abs(high-prev_close),abs(low-prev_close)))
atr=tr.rolling(w).mean()

garman_klass_volatility()

range_volatility.py

Brief: OHLC range-based volatility estimator.

Physical meaning: Combines high-low range and open-close movement to use more daily information than close returns alone.

Formula: $Var_{GK} = 0.5 \ln(H/L)^2 - (2\ln 2 - 1) \ln(C/O)^2$; $\sigma = \sqrt{\text{mean } Var_{GK}}$

Pros: Uses high/low/open information to estimate intraday variability more efficiently than close-only measures.

Cons: Sensitive to bad OHLC prints, overnight jumps and model assumptions; estimators are not interchangeable.

Needs: OHLC

Outputs: Garman Klass Volatility {window}

Call: garman_klass_volatility(df, windows=(20, 60, 120))

Python: v=.5*np.log(high/low)**2-(2*np.log(2)-1)*np.log(close/open_)**2
sigma=np.sqrt(v.clip(lower=0).rolling(w).mean())

rogers_satchell_volatility()

range_volatility.py

Brief: OHLC volatility estimator designed to tolerate non-zero price drift.

Physical meaning: Uses directional relationships among open, high, low and close to estimate intraday variance.

Formula: $RS = \ln(H/C) \ln(H/O) + \ln(L/C) \ln(L/O)$; $\sigma = \sqrt{\text{mean}(RS)}$

Pros: Uses high/low/open information to estimate intraday variability more efficiently than close-only measures.

Cons: Sensitive to bad OHLC prints, overnight jumps and model assumptions; estimators are not interchangeable.

Needs: OHLC

Outputs: Rogers Satchell Volatility {window}

Call: rogers_satchell_volatility(df, windows=(20, 60, 120))

Python: `rs=np.log(high/close)*np.log(high/open_)+np.log(low/close)*np.log(low/open_)`
`sigma=np.sqrt(rs.clip(lower=0).rolling(w).mean())`

trend_regime()

regimes.py

Brief: Binary +/-1 state based on short SMA versus long SMA.

Physical meaning: Classifies the price environment as broadly upward or downward trending.

Formula: $Regime = +1$ if $SMA_{short} > SMA_{long}$ else -1

Pros: Lets models condition behaviour on market state instead of assuming one relationship always holds.

Cons: Hard thresholds can be arbitrary and regimes can switch after the signal is observed.

Needs: Close

Outputs: Trend Regime {short_window} {long_window}

Call: trend_regime(df, short_window=50, long_window=200)

Python: `regime=np.where(close.rolling(s).mean()>close.rolling(l).mean(),1,-1)`

range_ratios()

range_volatility.py

Brief: Ratio of short-term average high-low range to long-term average range.

Physical meaning: Values above 1 indicate intraday movement has recently expanded relative to its baseline.

Formula: $RangeRatio(s, l) = \text{mean}_s(Range) / \text{mean}_l(Range)$

Pros: Uses high/low/open information to estimate intraday variability more efficiently than close-only measures.

Cons: Sensitive to bad OHLC prints, overnight jumps and model assumptions; estimators are not interchangeable.

Needs: High + Low + Close

Outputs: Range Ratio {short_window} {long_window}

Call: range_ratios(df, short_windows=(5, 20), long_windows=(20, 60))

Python: `ratio=rng.rolling(s).mean()/rng.rolling(l).mean()`

volatility_regime()

regimes.py

Brief: Short/long volatility ratio plus a high-volatility indicator.

Physical meaning: Shows whether recent realised risk is elevated relative to the stock's longer baseline.

Formula: $VolRegime = \sigma_{short} / \sigma_{long}$; $HighVol = 1[\sigma_{short} > \sigma_{long}]$

Pros: Lets models condition behaviour on market state instead of assuming one relationship always holds.

Cons: Hard thresholds can be arbitrary and regimes can switch after the signal is observed.

Needs: Close

Outputs: Volatility Regime {short_window} {long_window}; High Volatility Regime {short_window} {long_window}

Call: volatility_regime(df, short_window=20, long_window=252)

Python: `sv=r.rolling(s).std(); lv=r.rolling(l).std()`
`ratio=sv/lv; flag=(sv>lv).astype(int)`

drawdown_regime()

regimes.py

Brief: Indicator that current drawdown is below a selected loss threshold.

Physical meaning: Flags stressed states such as being more than 10% or 20% below a rolling peak.

Formula: $DDR_{Regime}=1[DD_w < threshold]$

Pros: Lets models condition behaviour on market state instead of assuming one relationship always holds.

Cons: Hard thresholds can be arbitrary and regimes can switch after the signal is observed.

Needs: Close

Outputs: Drawdown Regime {window} Below {label} Percent

Call: drawdown_regime(df, windows=(60, 252), thresholds=(-0.1, -0.2))

Python: dd=close/close.rolling(w).max()-1
flag=(dd<threshold).astype(int)

rolling_alpha()

residual.py

Brief: Rolling market-model intercept implied by stock and benchmark returns.

Physical meaning: Positive alpha means recent average stock return exceeded what its estimated beta and market return would imply.

Formula: $\alpha_w = \text{mean}_w(r_s) - \beta_w * \text{mean}_w(r_m)$

Pros: Separates market-driven movement from stock-specific behaviour and risk.

Cons: Depends on the benchmark model; rolling alpha/residuals can be unstable with short samples.

Needs: Close + market Close

Outputs: Rolling Alpha {window}

Call: rolling_alpha(df, windows=(20, 60, 120, 252), market_df=None, market_ticker='^GSPC')

Python: beta=r.rolling(w).cov(rm)/rm.rolling(w).var()
alpha=r.rolling(w).mean()-beta*rm.rolling(w).mean()

momentum_regime()

regimes.py

Brief: Indicator that cumulative momentum over a window is positive.

Physical meaning: Separates positive-trend from negative-trend states without using the exact return magnitude.

Formula: $Mom_{Regime}_w=1[Mom_w > 0]$

Pros: Lets models condition behaviour on market state instead of assuming one relationship always holds.

Cons: Hard thresholds can be arbitrary and regimes can switch after the signal is observed.

Needs: Close

Outputs: Positive Momentum Regime {window}

Call: momentum_regime(df, windows=(20, 60, 252))

Python: flag=(close.pct_change(w)>0).astype(int)

idiosyncratic_volatility()

residual.py

Brief: Volatility of returns left after removing estimated market exposure.

Physical meaning: Measures company-specific risk not explained by the broad market factor.

Formula: $\epsilon_t = r_s - \alpha - \beta r_m$; $IdioVol_w = Std_w(\epsilon)$

Pros: Separates market-driven movement from stock-specific behaviour and risk.

Cons: Depends on the benchmark model; rolling alpha/residuals can be unstable with short samples.

Needs: Close + market Close

Outputs: Idiosyncratic Volatility {window}

Call: idiosyncratic_volatility(df, windows=(20, 60, 252), market_df=None, market_ticker='^GSPC')

Python: resid=r-alpha-beta*rm
idio=resid.rolling(w).std()

residual_momentum()

residual.py

Brief: Compounded momentum of market-model residual returns.

Physical meaning: Measures stock-specific trend after stripping out broad market movement.

Formula: $ResMom_w = product_{j=1..w}(1+epsilon_j)-1$

Pros: Separates market-driven movement from stock-specific behaviour and risk.

Cons: Depends on the benchmark model; rolling alpha/residuals can be unstable with short samples.

Needs: Close + market Close

Outputs: Residual Momentum {window}

Call: residual_momentum(df, windows=(20, 60, 120, 252), market_df=None, market_ticker='^GSPC')

Python: resmom=(1+resid).rolling(w).apply(lambda x:x.prod(),raw=True)-1

return_lags()

returns.py

Brief: Lagged simple daily returns.

Physical meaning: Provides recent directional price changes without using future information.

Formula: $r_t=P_t/P_{(t-1)}-1$; $ReturnLag_k$ at $t = r_{(t-k+1)}$

Pros: Foundational price-change information used by nearly every time-series model.

Cons: Returns are very noisy and lag features usually have weak standalone predictive power.

Needs: Close

Outputs: Return Lag {period}

Call: return_lags(df, periods=(1, 2, 3, 5, 10, 20))

Python: r=close.pct_change()
lag=r.shift(k-1)

market_r_squared()

residual.py

Brief: Squared rolling stock-market correlation.

Physical meaning: Approximates the fraction of return variation explained by a one-factor market relationship.

Formula: $R2_w = Corr_w(r_s, r_m)^2$

Pros: Separates market-driven movement from stock-specific behaviour and risk.

Cons: Depends on the benchmark model; rolling alpha/residuals can be unstable with short samples.

Needs: Close + market Close

Outputs: Market R Squared {window}

Call: market_r_squared(df, windows=(20, 60, 120, 252), market_df=None, market_ticker='^GSPC')

Python: r2 = r.rolling(w).corr(rm) ** 2

log_return_lags()

returns.py

Brief: Lagged logarithmic daily returns.

Physical meaning: Log returns are additive through time and symmetric in compounding mathematics.

Formula: $ell_t=\ln(P_t/P_{(t-1)})$; $Lag_k=ell_{(t-k+1)}$

Pros: Foundational price-change information used by nearly every time-series model.

Cons: Returns are very noisy and lag features usually have weak standalone predictive power.

Needs: Close

Outputs: Log Return Lag {period}

Call: log_return_lags(df, periods=(1, 2, 5, 10, 20))

Python: lr=np.log(close/close.shift(1))
lag=lr.shift(k-1)

absolute_return_lags()

returns.py

Brief: Lagged absolute return magnitude.

Physical meaning: Measures recent shock size regardless of whether price rose or fell; useful for volatility forecasting.

Formula: $AbsLag_k = |r_{(t-k+1)}|$

Pros: Foundational price-change information used by nearly every time-series model.

Cons: Returns are very noisy and lag features usually have weak standalone predictive power.

Needs: Close

Outputs: Absolute Return Lag {period}

Call: absolute_return_lags(df, periods=(1, 2, 5, 10, 20))

Python: lag=close.pct_change().abs().shift(k-1)

rolling_mean_return()

returns.py

Brief: Rolling mean and median of daily returns.

Physical meaning: Mean captures average drift; median gives a more robust typical daily return.

Formula: $Mean_w = (1/w) \sum r$; $Median_w = median(r_{window})$

Pros: Foundational price-change information used by nearly every time-series model.

Cons: Returns are very noisy and lag features usually have weak standalone predictive power.

Needs: Close

Outputs: Mean Return {window}; Median Return {window}

Call: rolling_mean_return(df, windows=(5, 10, 20, 60, 120, 252))

Python: mean=r.rolling(w).mean(); median=r.rolling(w).median()

squared_return_lags()

returns.py

Brief: Lagged squared return.

Physical meaning: Emphasises large moves and acts as a simple proxy for recent variance shocks.

Formula: $SqLag_k = r_{(t-k+1)}^2$

Pros: Foundational price-change information used by nearly every time-series model.

Cons: Returns are very noisy and lag features usually have weak standalone predictive power.

Needs: Close

Outputs: Squared Return Lag {period}

Call: squared_return_lags(df, periods=(1, 2, 5, 10, 20))

Python: lag=(close.pct_change()²).shift(k-1)

positive_negative_days()

returns.py

Brief: Counts, ratios and conditional mean sizes of positive and negative days.

Physical meaning: Separates whether a trend came from many consistent moves or a few large days.

Formula: $PosRatio = \#(r > 0) / w$; $NegRatio = \#(r < 0) / w$; $MeanPos = mean(r | r > 0)$

Pros: Foundational price-change information used by nearly every time-series model.

Cons: Returns are very noisy and lag features usually have weak standalone predictive power.

Needs: Close

Outputs: Positive Days {window}; Negative Days {window}; Positive Day Ratio {window}; Negative Day Ratio {window}; Mean Positive Return {window}; Mean Negative Return {window}

Call: positive_negative_days(df, windows=(5, 10, 20, 60))

Python: pos=(r>0).rolling(w).sum(); neg=(r<0).rolling(w).sum()
mean_pos=r.where(r>0).rolling(w,min_periods=1).mean()

sector_relative_returns()

sector_relative.py

Brief: Stock return minus sector benchmark return.

Physical meaning: Positive values mean the stock outperformed its sector, not merely benefited from a sector-wide rally.

Formula: $SectorRelRet_p = R_{stock} - R_{sector}$

Pros: Controls for sector direction and helps identify stock-specific strength within peers.

Cons: Requires a sensible sector benchmark and can be unstable when sector definitions are broad.

Needs: Close + sector benchmark Close

Outputs: Sector Relative Return {period}

Call: sector_relative_returns(df, sector_df=None, sector_ticker=None, periods=(1, 5, 20, 60, 120, 252))

Python: rel=close.pct_change(p)-sector_close.pct_change(p)

sector_relative_ma_distance()

sector_relative.py

Brief: Stock MA distance minus sector MA distance.

Physical meaning: Measures whether the stock is more extended/stronger versus its own trend than its sector is.

Formula: $RelMADist = (P_s/MA_s - 1) - (P_{sec}/MA_{sec} - 1)$

Pros: Controls for sector direction and helps identify stock-specific strength within peers.

Cons: Requires a sensible sector benchmark and can be unstable when sector definitions are broad.

Needs: Close + sector benchmark Close

Outputs: Sector Relative MA Distance {window}

Call: sector_relative_ma_distance(df, sector_df=None, sector_ticker=None, windows=(20, 50, 200))

Python: rel=(close/close.rolling(w).mean()-1)-(sector/sector.rolling(w).mean()-1)

sector_relative_strength()

sector_relative.py

Brief: Momentum of the stock-to-sector price ratio.

Physical meaning: Tracks whether the stock is persistently gaining ground against sector peers.

Formula: $SectorRS_w = (P_{stock}/P_{sector})_t / (P_{stock}/P_{sector})_{(t-w)} - 1$

Pros: Controls for sector direction and helps identify stock-specific strength within peers.

Cons: Requires a sensible sector benchmark and can be unstable when sector definitions are broad.

Needs: Close + sector benchmark Close

Outputs: Sector Relative Strength {window}

Call: sector_relative_strength(df, sector_df=None, sector_ticker=None, windows=(20, 60, 120, 252))

Python: relative_price=close/sector_close
rs=relative_price.pct_change(w)

historical_var()

tail_risk.py

Brief: Rolling lower empirical return quantile (historical VaR threshold).

Physical meaning: A 5% VaR of -3% means only about 5% of recent daily returns were worse than -3%.

Formula: $VaR_q(w) = Q_q(r_{window})$

Pros: Focuses directly on extreme losses and asymmetric return tails.

Cons: Tail estimates need substantial data and are unstable in short windows.

Needs: Close

Outputs: VaR {label} Percent {window}

Call: historical_var(df, windows=(20, 60, 252), quantiles=(0.01, 0.05))

Python: var = r.rolling(w).quantile(q)

expected_shortfall()

tail_risk.py

Brief: Average return in the tail at or below the VaR cutoff.

Physical meaning: Measures the typical loss on the worst q fraction of recent days, not just the cutoff.

Formula: $ES_{(q,w)} = E[r \mid r \leq Q_q(r)]$

Pros: Focuses directly on extreme losses and asymmetric return tails.

Cons: Tail estimates need substantial data and are unstable in short windows.

Needs: Close

Outputs: Expected Shortfall {label} Percent {window}

Call: expected_shortfall(df, windows=(20, 60, 252), quantiles=(0.01, 0.05))

Python: es=r.rolling(w).apply(lambda x:x[x<=x.quantile(q)].mean(),raw=False)

best_worst_return()

tail_risk.py

Brief: Best and worst single daily return inside each lookback.

Physical meaning: Shows recent upside and downside extremes directly.

Formula: $Best_w = \max(r_{window}); Worst_w = \min(r_{window})$

Pros: Focuses directly on extreme losses and asymmetric return tails.

Cons: Tail estimates need substantial data and are unstable in short windows.

Needs: Close

Outputs: Best Return {window}; Worst Return {window}

Call: best_worst_return(df, windows=(5, 20, 60, 252))

Python: best=r.rolling(w).max(); worst=r.rolling(w).min()

extreme_returns()

tail_risk.py

Brief: Rolling frequency of returns more than k standard deviations from the local mean.

Physical meaning: Shows how often unusually large moves are occurring relative to recent volatility.

Formula: $ExtremeFreq = \text{mean}(1[|r - \mu_w| > k \sigma_w])$

Pros: Focuses directly on extreme losses and asymmetric return tails.

Cons: Tail estimates need substantial data and are unstable in short windows.

Needs: Close

Outputs: Extreme Return Frequency {window} {standard_deviation} SD

Call: extreme_returns(df, windows=(20, 60, 252), standard_deviations=(2, 3))

Python: extreme=(r-r.rolling(w).mean()).abs(>k*r.rolling(w).std())
freq=extreme.rolling(w).mean()

tail_ratio()

tail_risk.py

Brief: Ratio of upper-tail magnitude to lower-tail magnitude.

Physical meaning: Above 1 indicates recent upside tail is larger than downside tail in absolute percentile terms.

Formula: $TailRatio_w = |Q_{upper}| / |Q_{lower}|$

Pros: Focuses directly on extreme losses and asymmetric return tails.

Cons: Tail estimates need substantial data and are unstable in short windows.

Needs: Close

Outputs: Tail Ratio {window}

Call: tail_ratio(df, windows=(20, 60, 252), lower=0.05, upper=0.95)

Python: ratio=r.rolling(w).quantile(.95).abs()/r.rolling(w).quantile(.05).abs()

rsi()

technical.py

Brief: Wilder-style Relative Strength Index from exponentially averaged gains and losses.

Physical meaning: Values near 100 indicate gains dominate recent changes; near 0 indicate losses dominate.

Formula: $RS = EWMA(gains) / EWMA(losses)$; $RSI = 100 - 100 / (1 + RS)$

Pros: Compact transformations of trend, mean reversion and range position; easy to compute.

Cons: Many indicators are redundant transformations of price and are easy to overfit without OOS testing.

Needs: Close

Outputs: RSI {window}

Call: rsi(df, windows=(5, 10, 14, 20, 60))

Python: `rs=gains.ewm(alpha=1/w,adjust=False).mean()/losses.ewm(alpha=1/w,adjust=False).mean()
rsi=100-100/(1+rs)`

bollinger()

technical.py

Brief: Price z-score plus position and width of Bollinger-style bands.

Physical meaning: Shows how unusual price is versus its recent mean and how wide the recent price distribution is.

Formula: $z = (P - MA) / SD$; $Position = (P - Lower) / (Upper - Lower)$; $Width = (Upper - Lower) / MA$

Pros: Compact transformations of trend, mean reversion and range position; easy to compute.

Cons: Many indicators are redundant transformations of price and are easy to overfit without OOS testing.

Needs: Close

Outputs: Price Z Score {window}; Bollinger Position {window} {standard_deviation}; Bollinger Width {window} {standard_deviation}

Call: bollinger(df, windows=(20, 60), standard_deviations=(1, 2))

Python: `z=(close-mean)/std
pos=(close-(mean-k*std))/(2*k*std); width=(2*k*std)/mean`

macd()

technical.py

Brief: Normalised fast-minus-slow EMA, its signal EMA and histogram.

Physical meaning: Measures medium-term trend separation and whether that separation is accelerating or fading.

Formula: $MACD = (EMA_{fast} - EMA_{slow}) / P$; $Signal = EMA(MACD)$; $Hist = MACD - Signal$

Pros: Compact transformations of trend, mean reversion and range position; easy to compute.

Cons: Many indicators are redundant transformations of price and are easy to overfit without OOS testing.

Needs: Close

Outputs: MACD {fast_span} {slow_span}; MACD Signal {fast_span} {slow_span} {signal_span}; MACD Histogram {fast_span} {slow_span} {signal_span}

Call: macd(df, fast_spans=(12,), slow_spans=(26,), signal_spans=(9,))

Python: `value=(ema_fast-ema_slow)/close
signal=value.ewm(span=sig,adjust=False).mean(); hist=value-signal`

return_z_score()

technical.py

Brief: Current daily return standardised by its recent mean and volatility.

Physical meaning: Shows whether today's move is unusually large relative to the stock's own recent behaviour.

Formula: $z_r = (r_t - mean_w(r)) / std_w(r)$

Pros: Compact transformations of trend, mean reversion and range position; easy to compute.

Cons: Many indicators are redundant transformations of price and are easy to overfit without OOS testing.

Needs: Close

Outputs: Return Z Score {window}

Call: return_z_score(df, windows=(20, 60, 252))

Python: `z=(r-r.rolling(w).mean())/r.rolling(w).std()`

stochastic_oscillator()

technical.py

Brief: Close location between rolling low and rolling high, scaled 0-100.

Physical meaning: Values near 100 mean price closes near the top of its recent range; near 0 means near the bottom.

Formula: $K=100*(C-Low_w)/(High_w-Low_w)$

Pros: Compact transformations of trend, mean reversion and range position; easy to compute.

Cons: Many indicators are redundant transformations of price and are easy to overfit without OOS testing.

Needs: High + Low + Close

Outputs: Stochastic Oscillator {window}

Call: stochastic_oscillator(df, windows=(14, 20, 60))

Python: `k=100*(close-low.rolling(w).min()/(high.rolling(w).max()-low.rolling(w).min()))`

trend_r_squared()

trend.py

Brief: R-squared of a linear fit to log price.

Physical meaning: Near 1 means price followed a very smooth directional trend; low values mean a noisy path.

Formula: $R^2=1-SSE/SST$ for $\ln(P)$ versus time

Pros: Measures direction, smoothness and consistency rather than only endpoint return.

Cons: Trend estimates lag turning points and are sensitive to window choice.

Needs: Close

Outputs: Trend R Squared {window}

Call: trend_r_squared(df, windows=(20, 60, 120))

Python: `fit=slope*x+intercept`
`r2=1-((y-fit)**2).sum()/((y-y.mean())**2).sum()`

trend_slope()

trend.py

Brief: Linear-regression slope of log price through time.

Physical meaning: Positive slope means a rising exponential-like trend; magnitude represents average log-price change per step.

Formula: $\ln(P_j)=a+b_j$; $feature=b$

Pros: Measures direction, smoothness and consistency rather than only endpoint return.

Cons: Trend estimates lag turning points and are sensitive to window choice.

Needs: Close

Outputs: Trend Slope {window}

Call: trend_slope(df, windows=(10, 20, 60, 120))

Python: `slope=np.polyfit(np.arange(len(x)),np.log(x),1)[0]`

trend_strength()

trend.py

Brief: Trend slope multiplied by trend R-squared.

Physical meaning: Rewards trends that are both directional and smooth rather than large but erratic.

Formula: $TrendStrength_w = Slope_w * R^2_w$

Pros: Measures direction, smoothness and consistency rather than only endpoint return.

Cons: Trend estimates lag turning points and are sensitive to window choice.

Needs: Close

Outputs: Trend Strength {window}

Call: trend_strength(df, windows=(20, 60, 120))

Python: `strength=slope*r_squared`

trend_efficiency()

trend.py

Brief: Net absolute price displacement divided by total absolute path length.

Physical meaning: Near 1 means a smooth one-way move; near 0 means lots of back-and-forth noise.

Formula: $Efficiency_w = |P_t - P_{(t-w)}| / \sum_{j=1..w} |\Delta P_j|$

Pros: Measures direction, smoothness and consistency rather than only endpoint return.

Cons: Trend estimates lag turning points and are sensitive to window choice.

Needs: Close

Outputs: Trend Efficiency {window}

Call: trend_efficiency(df, windows=(5, 10, 20, 60, 120))

Python: direction=(close-close.shift(w)).abs()
path=close.diff().abs().rolling(w).sum(); eff=direction/path

rolling_volatility()

volatility.py

Brief: Rolling standard deviation of daily returns, optionally annualised.

Physical meaning: Measures recent realised risk; larger values mean day-to-day price changes have been wider.

Formula: $\sigma_w = Std_w(r)$; $annualised = \sigma_w * \sqrt{252}$

Pros: Volatility is persistent and often more forecastable than returns, making these strong risk features.

Cons: Backward-looking measures can react late to sudden shocks and may miss implied/forward information.

Needs: Close

Outputs: Volatility {window}

Call: rolling_volatility(df, windows=(3, 5, 10, 20, 40, 60, 90, 120, 252), annualize=False)

Python: vol=close.pct_change().rolling(w).std()*(np.sqrt(252) if annualize else 1)

trend_change()

trend.py

Brief: Change in log-price trend slope.

Physical meaning: Shows whether a trend is accelerating, flattening or reversing.

Formula: $\Delta Slope_w(p) = Slope_w(t) - Slope_w(t-p)$

Pros: Measures direction, smoothness and consistency rather than only endpoint return.

Cons: Trend estimates lag turning points and are sensitive to window choice.

Needs: Close

Outputs: Trend Slope {window} Change {period}

Call: trend_change(df, windows=(20, 60), periods=(5, 20))

Python: slope=log_price.rolling(w).apply(_slope, raw=True)
change=slope.diff(p)

ewma_volatility()

volatility.py

Brief: Exponentially weighted standard deviation of returns.

Physical meaning: Responds more quickly to new shocks because recent observations receive greater weight.

Formula: $\sigma_{EWMA} = EW\ weighted\ Std(r)$

Pros: Volatility is persistent and often more forecastable than returns, making these strong risk features.

Cons: Backward-looking measures can react late to sudden shocks and may miss implied/forward information.

Needs: Close

Outputs: EWMA Volatility {span}

Call: ewma_volatility(df, spans=(5, 10, 20, 60), annualize=False)

Python: vol=r.ewm(span=s, adjust=False).std()

volatility_ratios()

volatility.py

Brief: Ratio of short-horizon to long-horizon realised volatility.

Physical meaning: Above 1 means recent risk has expanded relative to the longer baseline; below 1 means compression.

Formula: $VolRatio(s, l) = \sigma_s / \sigma_l$

Pros: Volatility is persistent and often more forecastable than returns, making these strong risk features.

Cons: Backward-looking measures can react late to sudden shocks and may miss implied/forward information.

Needs: Close

Outputs: Volatility Ratio {short_window} {long_window}

Call: volatility_ratios(df, short_windows=(5, 10, 20), long_windows=(20, 60, 120, 252))

Python: ratio=r.rolling(s).std()/r.rolling(l).std()

semivariance()

volatility.py

Brief: Average squared positive and negative returns and downside/upside ratio.

Physical meaning: Measures direction-specific second-moment risk without subtracting a conditional mean.

Formula: $SemiDown = mean(r^2 | r < 0)$; $SemiUp = mean(r^2 | r > 0)$

Pros: Volatility is persistent and often more forecastable than returns, making these strong risk features.

Cons: Backward-looking measures can react late to sudden shocks and may miss implied/forward information.

Needs: Close

Outputs: Downside Semivariance {window}; Upside Semivariance {window}; Semivariance Ratio {window}

Call: semivariance(df, windows=(20, 60, 120))

Python: down=(r.where(r<0)**2).rolling(w,min_periods=m).mean()

up=(r.where(r>0)**2).rolling(w,min_periods=m).mean()

upside_downside_volatility()

volatility.py

Brief: Volatility estimated separately from positive and negative return observations plus their ratio.

Physical meaning: Shows whether recent downside moves are more erratic than upside moves.

Formula: $\sigma_{up} = Std(r | r > 0)$; $\sigma_{down} = Std(r | r < 0)$; $ratio = \sigma_{down} / \sigma_{up}$

Pros: Volatility is persistent and often more forecastable than returns, making these strong risk features.

Cons: Backward-looking measures can react late to sudden shocks and may miss implied/forward information.

Needs: Close

Outputs: Upside Volatility {window}; Downside Volatility {window}; Downside Upside Volatility Ratio {window}

Call: upside_downside_volatility(df, windows=(5, 20, 60, 120))

Python: up=r.where(r>0).rolling(w,min_periods=m).std()

down=r.where(r<0).rolling(w,min_periods=m).std()

volatility_change()

volatility.py

Brief: Change in rolling volatility over time.

Physical meaning: Positive values indicate risk is expanding; negative values indicate volatility is calming.

Formula: $\Delta \sigma(w, p, t) = \sigma(w, t) - \sigma(w, t - p)$

Pros: Volatility is persistent and often more forecastable than returns, making these strong risk features.

Cons: Backward-looking measures can react late to sudden shocks and may miss implied/forward information.

Needs: Close

Outputs: Volatility {window} Change {period}

Call: volatility_change(df, windows=(5, 20, 60, 252), periods=(1, 5, 20))

Python: vol=r.rolling(w).std()

change=vol.diff(p)

volatility_persistence()

volatility.py

Brief: Autocorrelation of absolute and squared returns.

Physical meaning: High values indicate volatility clustering: large moves tend to be followed by large moves.

Formula: $Corr_w(|r_t|, |r_{(t-k)}|)$ and $Corr_w(r_t^2, r_{(t-k)}^2)$

Pros: Volatility is persistent and often more forecastable than returns, making these strong risk features.

Cons: Backward-looking measures can react late to sudden shocks and may miss implied/forward information.

Needs: Close

Outputs: Absolute Return Autocorrelation {window} Lag {lag}; Squared Return Autocorrelation {window} Lag {lag}

Call: volatility_persistence(df, windows=(20, 60, 252), lags=(1, 5))

Python: `a=r.abs().rolling(w).corr(r.abs().shift(k))`
`s=(r**2).rolling(w).corr((r**2).shift(k))`

average_volume()

volume.py

Brief: Rolling average volume and volatility of percentage volume changes.

Physical meaning: Captures normal trading activity and how unstable that activity has been.

Formula: $AvgVol_w = mean_w(V)$; $VolVol_w = Std_w(Delta V/V)$

Pros: Adds participation and trading-intensity information to price signals.

Cons: Volume is exchange/data-source dependent and does not directly equal informed trading or liquidity.

Needs: Volume

Outputs: Average Volume {window}; Volume Volatility {window}

Call: average_volume(df, windows=(5, 20, 60, 252))

Python: `avg=volume.rolling(w).mean()`
`volvol=volume.pct_change().rolling(w).std()`

volume_changes()

volume.py

Brief: Percentage change in raw trading volume.

Physical meaning: Shows whether participation has increased or decreased relative to prior periods.

Formula: $VolChange_p = V_t/V_{(t-p)} - 1$

Pros: Adds participation and trading-intensity information to price signals.

Cons: Volume is exchange/data-source dependent and does not directly equal informed trading or liquidity.

Needs: Volume

Outputs: Volume Change {period}

Call: volume_changes(df, periods=(1, 5, 20))

Python: `change=df["Volume"].pct_change(p)`

relative_volume()

volume.py

Brief: Current volume relative to its rolling mean plus a volume z-score.

Physical meaning: Shows whether today's trading activity is unusually heavy or light for this stock.

Formula: $RelVol = V/mean_w(V)$; $z_V = (V - mean_w(V))/std_w(V)$

Pros: Adds participation and trading-intensity information to price signals.

Cons: Volume is exchange/data-source dependent and does not directly equal informed trading or liquidity.

Needs: Volume

Outputs: Relative Volume {window}; Volume Z Score {window}

Call: relative_volume(df, windows=(5, 20, 60))

Python: `avg=volume.rolling(w).mean(); std=volume.rolling(w).std()`
`rel=volume/avg; z=(volume-avg)/std`

volume_momentum()

volume.py

Brief: Short average volume relative to long average volume.

Physical meaning: Positive values indicate trading activity has recently increased versus its longer baseline.

Formula: $VolMom(s, l) = AvgVol_s / AvgVol_l - 1$

Pros: Adds participation and trading-intensity information to price signals.

Cons: Volume is exchange/data-source dependent and does not directly equal informed trading or liquidity.

Needs: Volume

Outputs: Volume Momentum {short_window} {long_window}

Call: volume_momentum(df, short_windows=(5, 20), long_windows=(20, 60, 252))

Python: mom=volume.rolling(s).mean()/volume.rolling(l).mean()-1

return_volume_interactions()

volume.py

Brief: Return/momentum multiplied by relative volume.

Physical meaning: A move receives a larger magnitude when accompanied by unusually strong trading participation.

Formula: $I(m, v) = R_m * V / AvgVol_v$

Pros: Adds participation and trading-intensity information to price signals.

Cons: Volume is exchange/data-source dependent and does not directly equal informed trading or liquidity.

Needs: Close + Volume

Outputs: Return Volume Interaction {momentum_window} {volume_window}

Call: return_volume_interactions(df, momentum_windows=(1, 5, 20, 60), volume_windows=(20, 60))

Python: interaction=close.pct_change(m)*(volume/volume.rolling(v).mean())

TARGET REFERENCE

first_hit_barrier()

barriers.py

Brief: Path label for whether an upper or lower fixed return barrier is hit first.

Physical meaning: Distinguishes a path that reaches profit before loss from one that reaches loss first, even if endpoints match.

Formula: $y=+1$ if $+u$ hit before l ; -1 if l hit before $+u$; 0 if neither/tie within h

Pros: Captures path ordering and event-style outcomes rather than endpoint return only.

Cons: Computationally heavier; barrier sizes/horizons can create severe class imbalance or data-mining risk.

Needs: Close

Outputs: Barrier {up_label} {down_label} {horizon}

Call: first_hit_barrier(df, upper=(0.02, 0.05), lower=(-0.02, -0.05), horizons=(10, 20, 60))

Python: path=prices[i+1:i+h+1]/prices[i]-1
upper=np.where(path>=u)[0]; lower=np.where(path<=l)[0]

future_direction()

direction.py

Brief: Binary label for whether horizon return is positive.

Physical meaning: Answers the simplest directional question: did price finish above today's level?

Formula: $y=1[R_-(t, t+h)>0]$

Pros: Transforms noisy returns into interpretable classification problems.

Cons: Threshold choices affect class balance and discard information about return magnitude.

Needs: Close

Outputs: Future Direction {horizon}

Call: future_direction(df, horizons=(1, 5, 20, 60))

Python: forward=close.shift(-h)/close-1
y=(forward>0).astype(float)

volatility_barrier()

barriers.py

Brief: First-hit label with barriers scaled by current volatility and sqrt(horizon).

Physical meaning: Makes event thresholds adapt to each stock's current risk instead of using the same percentage for all.

Formula: $scale=\sigma_w, t*\sqrt{h}$; $upper=k_u*scale$; $lower=-k_l*scale$; first hit defines class

Pros: Captures path ordering and event-style outcomes rather than endpoint return only.

Cons: Computationally heavier; barrier sizes/horizons can create severe class imbalance or data-mining risk.

Needs: Close

Outputs: Volatility Barrier {volatility_window} {horizon} {upper_multiple} {lower_multiple}

Call: volatility_barrier(df, horizons=(20, 60), volatility_windows=(20, 60), upper_multiples=(1, 2), lower_multiples=(1, 2))

Python: scale=volatility[i]*np.sqrt(h)
upper=k_up*scale; lower=-k_down*scale

threshold_direction()

direction.py

Brief: Binary label for whether future return exceeds a positive threshold.

Physical meaning: Focuses on economically meaningful upside rather than tiny positive moves.

Formula: $y=1[R_-(t, t+h)>c]$

Pros: Transforms noisy returns into interpretable classification problems.

Cons: Threshold choices affect class balance and discard information about return magnitude.

Needs: Close

Outputs: column

Call: threshold_direction(df, horizons=(5, 20, 60), thresholds=(0.01, 0.02, 0.05, 0.1))

Python: y=(forward>threshold).astype(float)

three_class_direction()

direction.py

Brief: Three-class label: meaningful rise, neutral, or meaningful fall.

Physical meaning: Separates strong upside/downside outcomes from noisy movement around zero.

Formula: $y=+1$ if $R>c$; 0 if $|R|\leq c$; -1 if $R<-c$

Pros: Transforms noisy returns into interpretable classification problems.

Cons: Threshold choices affect class balance and discard information about return magnitude.

Needs: Close

Outputs: column

Call: three_class_direction(df, horizons=(20, 60), thresholds=(0.02, 0.05))

Python: $y=0$; $y[\text{forward}>c]=1$; $y[\text{forward}<-c]=-1$

future_minimum_return()

drawdown.py

Brief: Minimum return from today reached anywhere in the future horizon.

Physical meaning: Targets the worst mark-to-market loss relative to today's entry price.

Formula: $\text{FutureMinRet}_h = \min_{1 \leq j \leq h} (P_{t+j} / P_{t-1})$

Pros: Targets future downside path risk directly rather than only final return.

Cons: Path-dependent, noisy and overlapping; definitions of drawdown must be interpreted carefully.

Needs: Close

Outputs: Future Minimum Return {horizon}

Call: future_minimum_return(df, horizons=(5, 20, 60))

Python: $\text{path} = \text{prices}[i+1:i+h+1] / \text{prices}[i]-1$
 $\text{target} = \text{np.min}(\text{path})$

future_maximum_drawdown()

drawdown.py

Brief: Worst peak-to-trough drawdown occurring from today through the future horizon.

Physical meaning: Targets how painful the future price path becomes even if it later recovers.

Formula: $\text{FutureMDD}_h = \min_j (P_{t+j} / \text{running_peak}_{t:t+j}) - 1$

Pros: Targets future downside path risk directly rather than only final return.

Cons: Path-dependent, noisy and overlapping; definitions of drawdown must be interpreted carefully.

Needs: Close

Outputs: Future Maximum Drawdown {horizon}

Call: future_maximum_drawdown(df, horizons=(20, 60, 120))

Python: $\text{path} = \text{prices}[i:i+h+1]$
 $\text{peak} = \text{np.maximum.accumulate}(\text{path})$; $\text{target} = \text{np.min}(\text{path}/\text{peak}-1)$

future_drawdown_at_horizon()

drawdown.py

Brief: Future horizon price relative to the highest price between now and that horizon.

Physical meaning: Measures how far the horizon endpoint sits below the best price available during the intervening path.

Formula: $\text{DDatH}_h = P_{t+h} / \max(P_{t-1}, \dots, P_{t+h-1}) - 1$

Pros: Targets future downside path risk directly rather than only final return.

Cons: Path-dependent, noisy and overlapping; definitions of drawdown must be interpreted carefully.

Needs: Close

Outputs: Future Drawdown At Horizon {horizon}

Call: future_drawdown_at_horizon(df, horizons=(5, 20, 60))

Python: $\text{future} = \text{close}.\text{shift}(-h)$
 $\text{future_high} = \text{close}[:, -1].\text{rolling}(h+1).\text{max}()[::-1]$
 $\text{target} = \text{future}/\text{future_high}-1$

maximum_favourable_excursion()

excursions.py

Brief: Largest future return reached at any point inside the horizon.

Physical meaning: Measures best available upside after the observation date, not just the endpoint.

Formula: $MFE_h = \max_{\{1 \leq j \leq h\}} (P_{t+j} / P_t - 1)$

Pros: Measures best/worst path outcomes and timing, useful for entry/exit and risk research.

Cons: Uses future path information only suitable as a target; highly sensitive to the chosen horizon.

Needs: Close

Outputs: Maximum Favourable Excursion {horizon}

Call: maximum_favourable_excursion(df, horizons=(5, 20, 60))

Python: path=prices[i+1:i+h+1]/prices[i]-1
mfe=np.max(path)

time_to_favourable_excursion()

excursions.py

Brief: Number of periods until the maximum favourable excursion occurs.

Physical meaning: Shows how quickly the best upside opportunity appears after the signal.

Formula: $T_{MFE} = \arg\max_j \text{future_return}_j + 1$

Pros: Measures best/worst path outcomes and timing, useful for entry/exit and risk research.

Cons: Uses future path information only suitable as a target; highly sensitive to the chosen horizon.

Needs: Close

Outputs: Time To Maximum Favourable Excursion {horizon}

Call: time_to_favourable_excursion(df, horizons=(20, 60))

Python: time=np.argmax(path)+1

maximum_adverse_excursion()

excursions.py

Brief: Worst future return reached inside the horizon.

Physical meaning: Measures the deepest adverse move an investor would experience before the horizon ends.

Formula: $MAE_h = \min_{\{1 \leq j \leq h\}} (P_{t+j} / P_t - 1)$

Pros: Measures best/worst path outcomes and timing, useful for entry/exit and risk research.

Cons: Uses future path information only suitable as a target; highly sensitive to the chosen horizon.

Needs: Close

Outputs: Maximum Adverse Excursion {horizon}

Call: maximum_adverse_excursion(df, horizons=(5, 20, 60))

Python: path=prices[i+1:i+h+1]/prices[i]-1
mae=np.min(path)

time_to_adverse_excursion()

excursions.py

Brief: Number of periods until the maximum adverse excursion occurs.

Physical meaning: Shows how quickly the worst downside appears after the signal.

Formula: $T_{MAE} = \arg\min_j \text{future_return}_j + 1$

Pros: Measures best/worst path outcomes and timing, useful for entry/exit and risk research.

Cons: Uses future path information only suitable as a target; highly sensitive to the chosen horizon.

Needs: Close

Outputs: Time To Maximum Adverse Excursion {horizon}

Call: time_to_adverse_excursion(df, horizons=(20, 60))

Python: time=np.argmin(path)+1

future_return_rank()

ranking.py

Brief: Percentile rank of future return among stocks on the same date.

Physical meaning: Directly answers which stocks will outperform the rest of the investable universe over the horizon.

Formula: $Rank_i, t = percentile_rank_i(R_i, t \rightarrow t+h)$

Pros: Matches the stock-selection task: identify which stocks outperform peers on the same date.

Cons: Requires a sufficiently large, stable universe and careful handling of delisted/missing securities.

Needs: Long-form panel with Date/Ticker/Close

Outputs: Forward Return {horizon}; Future Return Rank {horizon}

Call: future_return_rank(df, horizons=(5, 20, 60), ticker_col='Ticker', date_col='Date', price_col='Close')

Python: forward=df.groupby(ticker)[price].shift(-h)/df[price]-1
rank=df.assign(__Target=forward).groupby(date)["__Target"].rank(pct=True)

bottom_quantile_label()

ranking.py

Brief: Binary label for whether a stock finishes in the bottom future-return fraction.

Physical meaning: Identifies likely future laggards and can support avoidance or long-short research.

Formula: $y = 1[FutureReturnRank \leq q]$

Pros: Matches the stock-selection task: identify which stocks outperform peers on the same date.

Cons: Requires a sufficiently large, stable universe and careful handling of delisted/missing securities.

Needs: Long-form panel with Date/Ticker/Close

Outputs: column

Call: bottom_quantile_label(df, horizons=(20, 60), quantiles=(0.2, 0.25), ticker_col='Ticker', date_col='Date', price_col='Close')

Python: y=(rank<=quantile).astype(float)

top_quantile_label()

ranking.py

Brief: Binary label for whether a stock finishes in the top chosen future-return fraction.

Physical meaning: Turns stock selection into classification of likely future leaders.

Formula: $y = 1[FutureReturnRank \geq 1-q]$

Pros: Matches the stock-selection task: identify which stocks outperform peers on the same date.

Cons: Requires a sufficiently large, stable universe and careful handling of delisted/missing securities.

Needs: Long-form panel with Date/Ticker/Close

Outputs: column

Call: top_quantile_label(df, horizons=(20, 60), quantiles=(0.2, 0.25), ticker_col='Ticker', date_col='Date', price_col='Close')

Python: y=(rank>=1-quantile).astype(float)

forward_return()

returns.py

Brief: Future simple return over each prediction horizon.

Physical meaning: The direct percentage gain/loss realised from today to the future horizon.

Formula: $R_-(t, t+h) = P_-(t+h) / P_-t - 1$

Pros: Directly aligns the model with future performance and supports regression or ranking.

Cons: Future returns are extremely noisy and overlapping horizons require careful purging/validation.

Needs: Close

Outputs: Forward Return {horizon}

Call: forward_return(df, horizons=(1, 5, 10, 20, 60, 120, 252))

Python: target=close.shift(-h)/close-1

forward_log_return()

returns.py

Brief: Future logarithmic return.

Physical meaning: Measures future compounded price change on an additive log scale.

Formula: $L_{-}(t, t+h) = \ln(P_{-}(t+h)/P_{-}t)$

Pros: Directly aligns the model with future performance and supports regression or ranking.

Cons: Future returns are extremely noisy and overlapping horizons require careful purging/validation.

Needs: Close

Outputs: Forward Log Return {horizon}

Call: forward_log_return(df, horizons=(1, 5, 20, 60, 120, 252))

Python: target=np.log(close.shift(-h)/close)

future_return_volatility_ratio()

risk_adjusted.py

Brief: Future return divided by realised future volatility.

Physical meaning: Rewards high future return obtained with a relatively calm path.

Formula: $Target_h = R_{-}(t, t+h) / FutureVol_h$

Pros: Optimises toward reward per unit of future risk rather than raw return alone.

Cons: Ratios can explode when risk is near zero and combine estimation errors from numerator and denominator.

Needs: Close

Outputs: Future Return Volatility Ratio {horizon}

Call: future_return_volatility_ratio(df, horizons=(5, 20, 60))

Python: forward=close.shift(-h)/close-1

vol=r.shift(-1)[::-1].rolling(h).std()[::-1]

target=forward/vol

forward_excess_return()

returns.py

Brief: Future stock return minus future benchmark return.

Physical meaning: Targets stock-specific outperformance rather than whether the whole market rises.

Formula: $Excess_h = R_{-}stock, h - R_{-}benchmark, h$

Pros: Directly aligns the model with future performance and supports regression or ranking.

Cons: Future returns are extremely noisy and overlapping horizons require careful purging/validation.

Needs: Close + benchmark Close

Outputs: Forward Excess Return {horizon}

Call: forward_excess_return(df, horizons=(5, 20, 60), benchmark_df=None, benchmark_ticker='^GSPC')

Python: target=(close.shift(-h)/close-1)-(bench.shift(-h)/bench-1)

future_sortino_ratio()

risk_adjusted.py

Brief: Future return divided by future downside volatility.

Physical meaning: Rewards future return while penalising only variability on negative-return days.

Formula: $Target_h = R_{-}(t, t+h) / sigma_down, future$

Pros: Optimises toward reward per unit of future risk rather than raw return alone.

Cons: Ratios can explode when risk is near zero and combine estimation errors from numerator and denominator.

Needs: Close

Outputs: Future Sortino Ratio {horizon}

Call: future_sortino_ratio(df, horizons=(20, 60))

Python: down=r.shift(-1)[::-1].where(lambda x:x<0).rolling(h,min_periods=m).std()[::-1]

target=forward/down

future_return_minus_risk()

risk_adjusted.py

Brief: Future return minus a chosen multiple of future volatility.

Physical meaning: Creates a utility-style target where return is rewarded and risk is explicitly penalised.

Formula: $Target_h, \lambda = R_-(t, t+h) - \lambda * FutureVol_h$

Pros: Optimises toward reward per unit of future risk rather than raw return alone.

Cons: Ratios can explode when risk is near zero and combine estimation errors from numerator and denominator.

Needs: Close

Outputs: Future Return Minus Risk {horizon} {risk_weight}

Call: future_return_minus_risk(df, horizons=(20, 60), risk_weights=(0.5, 1, 2))

Python: target=forward-risk_weight*future_vol

future_volatility()

volatility.py

Brief: Standard deviation of returns from t+1 through t+h.

Physical meaning: Targets the amount of realised risk the stock will experience over the coming horizon.

Formula: $FutureVol_h = Std(r_-(t+1), \dots, r_-(t+h))$

Pros: Volatility targets are economically useful and typically more predictable than raw returns.

Cons: Realised future volatility depends on horizon and estimator; overlapping labels create dependence.

Needs: Close

Outputs: Future Volatility {horizon}

Call: future_volatility(df, horizons=(5, 10, 20, 60), annualize=False)

Python: target=r.shift(-1)[::-1].rolling(h).std()[::-1]

future_return_drawdown_ratio()

risk_adjusted.py

Brief: Future horizon return divided by absolute future maximum drawdown.

Physical meaning: Rewards paths that deliver return efficiently relative to the worst peak-to-trough loss.

Formula: $Target_h = R_h / |FutureMDD_h|$

Pros: Optimises toward reward per unit of future risk rather than raw return alone.

Cons: Ratios can explode when risk is near zero and combine estimation errors from numerator and denominator.

Needs: Close

Outputs: Future Return Drawdown Ratio {horizon}

Call: future_return_drawdown_ratio(df, horizons=(20, 60))

Python: mdd=abs(np.min(path/np.maximum.accumulate(path)-1))
target=forward_return/mdd if mdd!=0 else np.nan

future_variance()

volatility.py

Brief: Variance of returns over the future horizon.

Physical meaning: Same risk concept as volatility but on the squared scale used by many statistical models.

Formula: $FutureVar_h = Var(r_-(t+1), \dots, r_-(t+h))$

Pros: Volatility targets are economically useful and typically more predictable than raw returns.

Cons: Realised future volatility depends on horizon and estimator; overlapping labels create dependence.

Needs: Close

Outputs: Future Variance {horizon}

Call: future_variance(df, horizons=(5, 20, 60))

Python: target=r.shift(-1)[::-1].rolling(h).var()[::-1]

future_upside_downside_volatility()

volatility.py

Brief: Future volatility split by positive and negative returns plus their ratio.

Physical meaning: Targets whether upcoming risk is likely to be concentrated more on downside or upside days.

Formula: *Future sigma_up=Std(r>0); sigma_down=Std(r<0); ratio=down/up over t+1:t+h*

Pros: Volatility targets are economically useful and typically more predictable than raw returns.

Cons: Realised future volatility depends on horizon and estimator; overlapping labels create dependence.

Needs: Close

Outputs: Future Upside Volatility {horizon}; Future Downside Volatility {horizon}; Future Downside Upside Volatility Ratio {horizon}

Call: future_upside_downside_volatility(df, horizons=(10, 20, 60))

Python: x=r.shift(-1)[::-1]
up=x.where(x>0).rolling(h,min_periods=m).std()[::-1]
down=x.where(x<0).rolling(h,min_periods=m).std()[::-1]

future_absolute_return()

volatility.py

Brief: Future mean and maximum absolute daily return.

Physical meaning: Targets typical future move size and the largest shock within the horizon regardless of direction.

Formula: *MeanAbs_h=mean(|r|); MaxAbs_h=max(|r|), future window*

Pros: Volatility targets are economically useful and typically more predictable than raw returns.

Cons: Realised future volatility depends on horizon and estimator; overlapping labels create dependence.

Needs: Close

Outputs: Future Mean Absolute Return {horizon}; Future Maximum Absolute Return {horizon}

Call: future_absolute_return(df, horizons=(5, 20, 60))

Python: a=r.abs().shift(-1)[::-1]
mean=a.rolling(h).mean()[::-1]; maximum=a.rolling(h).max()[::-1]

Suggested use inside the Equity Selector

1. Generate candidate feature families only from information known at time t .
2. Add one target family at a time (return, volatility, direction, barrier or cross-sectional rank).
3. Screen redundancy/stability inside training data only; do not let the final test period influence feature selection.
4. Evaluate both prediction metrics and portfolio outcomes. A weak return R-squared can still produce useful ranking information.
5. Treat experimental and technical features as candidates, not assumptions: keep them only if they survive out-of-sample tests.